

Verification of Distributed Protocols: Decidable Modeling and Invariant Inference

Oded Padon
VMware Research

Tutorial at FMCAD 2022
18 October 2022

Verification of Distributed Protocols: Decidable Modeling and Invariant Inference

Part 1: Decidable Modeling

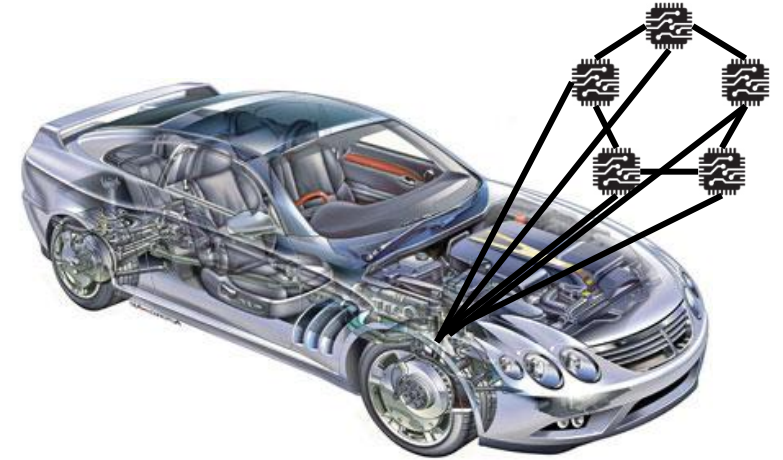
- The Ivy deductive verification system
- The many-sorted EPR fragment
- Main challenge: expressing interesting systems and properties in EPR
 - Expressing transitive closure
 - Expressing sets and cardinalities
 - Liveness and temporal verification

Part 2: Invariant Inference

- Problem setting

Distributed protocols – excellent opportunity for verification

- Distributed systems are everywhere
 - Safety-critical systems
 - Internet scale services
 - Cloud infrastructure
- Distributed systems are notoriously hard to get right
 - Even small protocols can be tricky
 - Bugs occur in rare scenarios
 - Testing is costly and not sufficient



SOSP'07

Best Paper Award

Zyzzyva: Speculative Byzantine Fault Tolerance

Zyzzyva is a state machine replication protocol based on three sub-protocols: (1) agreement, (2) view change, and (3) checkpoint. The *agreement* protocol orders requests for execution by the replicas. The *view change* protocol coordinates election of a new primary when the current primary is

CACM'08

Zyzzyva: Speculative Byzantine Fault Tolerance

ACM Transactions on Computer Systems '08

Zyzzyva: Speculative Byzantine Fault Tolerance

arXiv:1712.01367 [cs.DC] 2017

Revisiting Fast Practical Byzantine Fault Tolerance

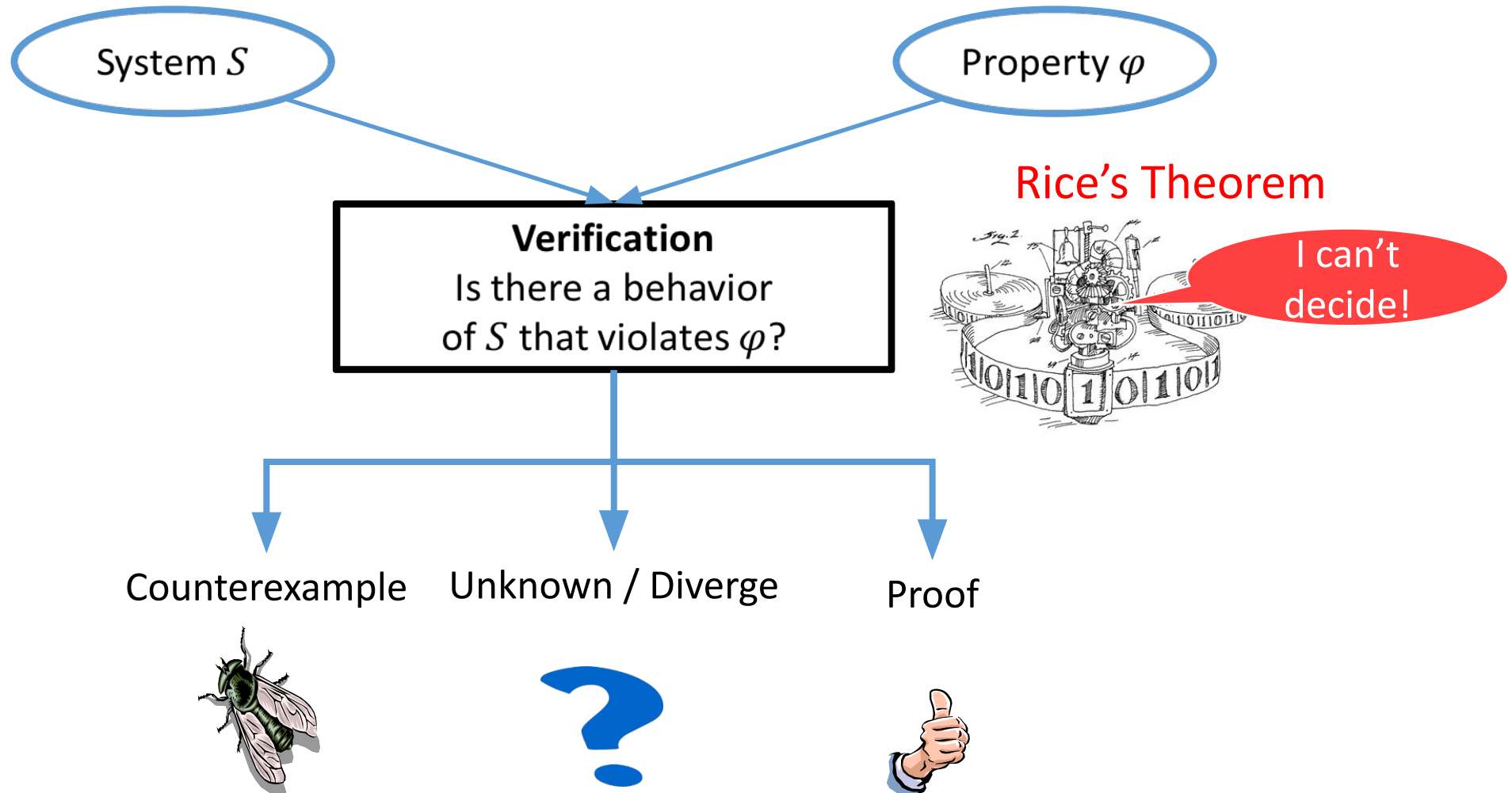
We now proceed to demonstrate that the view-change mechanism in Zyzzyva does not guarantee safety.

ASPLOS'16

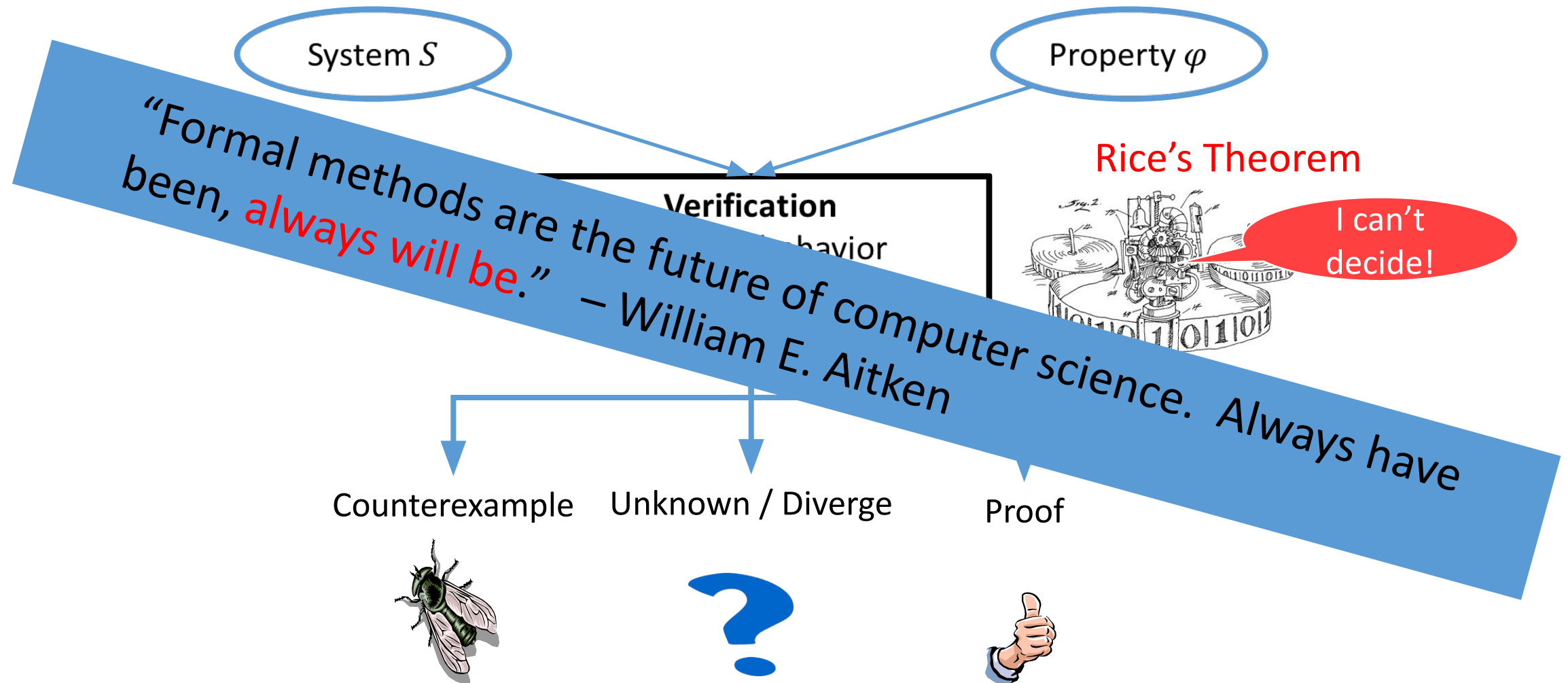
TaxDC: A Taxonomy of Non-Deterministic Concurrency Bugs in Datacenter Distributed Systems

We present TaxDC, the largest and most comprehensive taxonomy of non-deterministic concurrency bugs in distributed systems. We study 104 distributed concurrency (DC) bugs from four widely-deployed cloud-scale datacenter distributed systems, Cassandra, Hadoop MapReduce, HBase and ZooKeeper. We study DC-bug characteristics along sev-

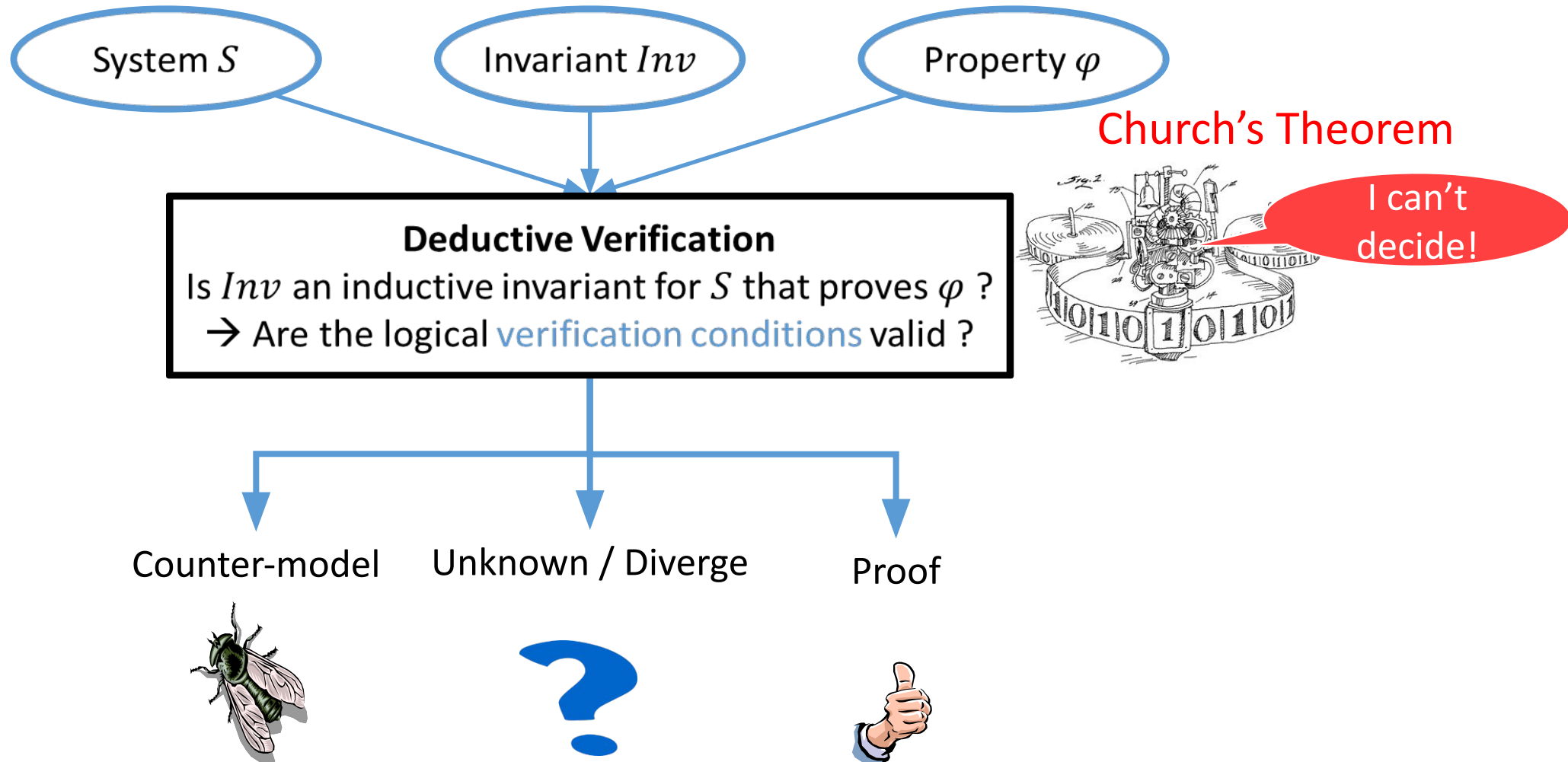
Automatic verification of infinite-state systems



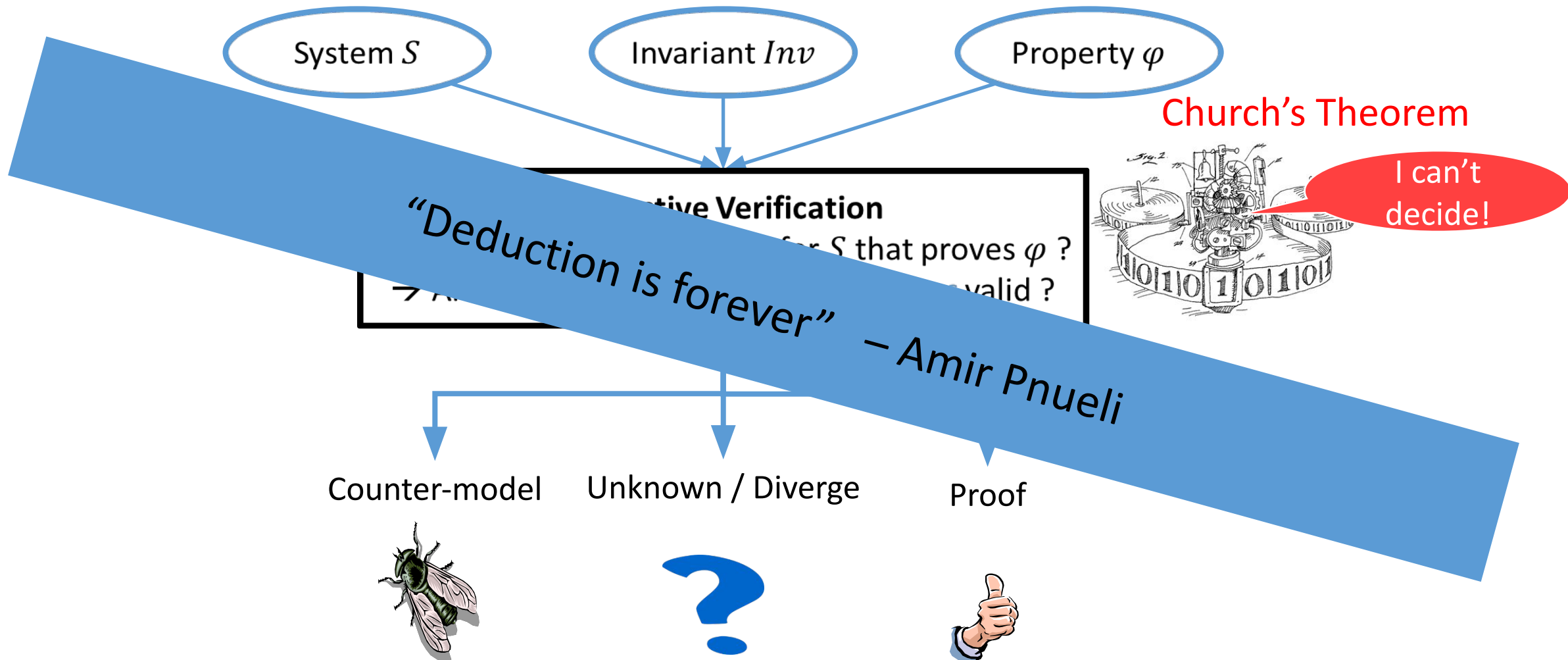
Automatic verification of infinite-state systems



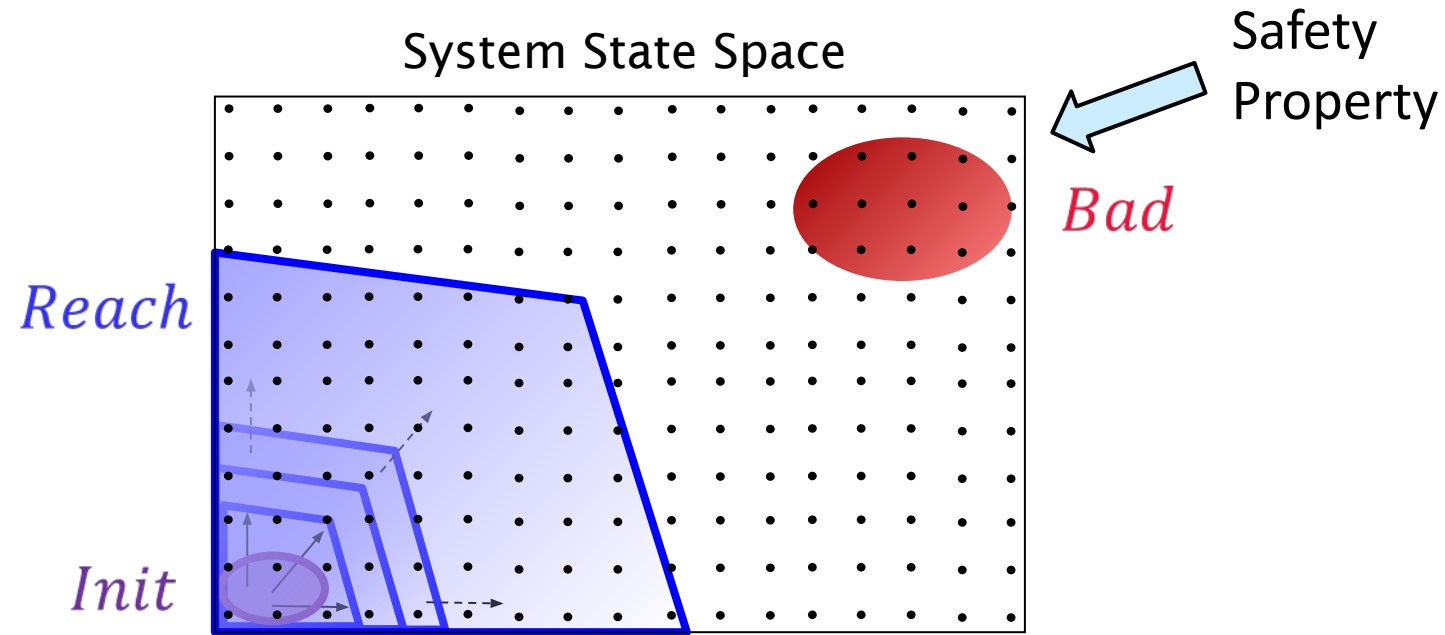
Deductive verification



Deductive verification

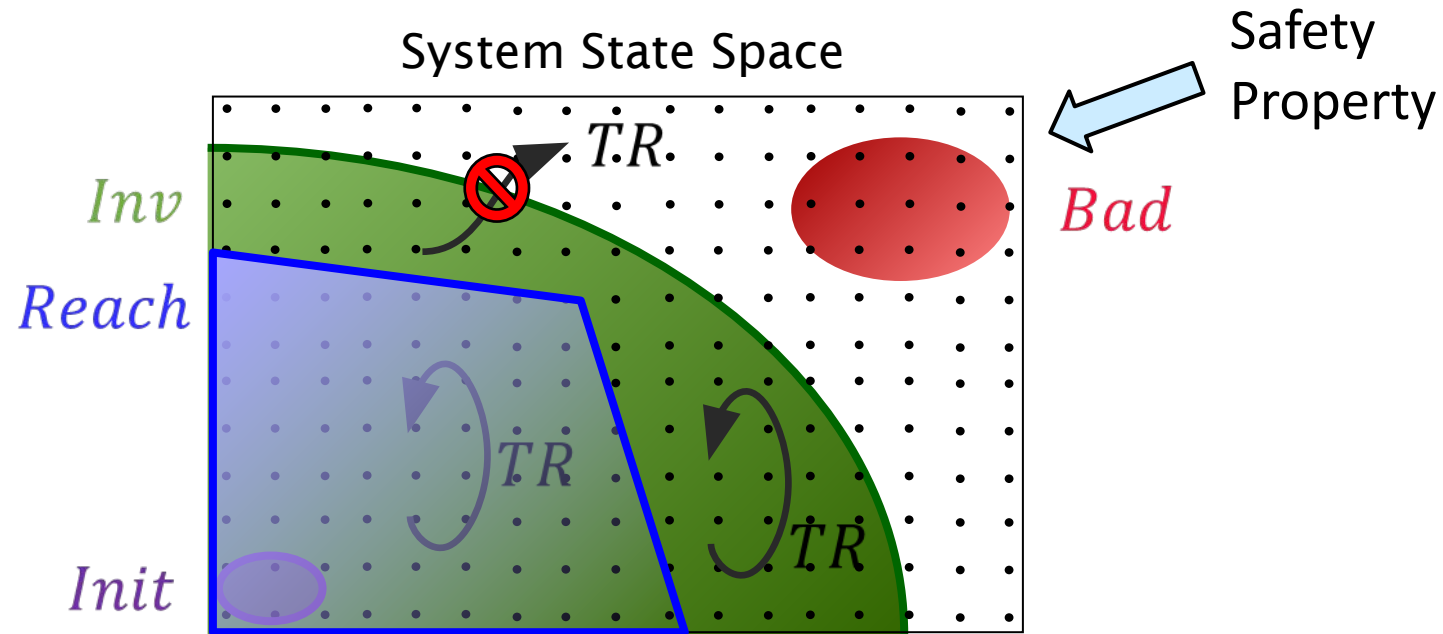


Inductive invariants



System S is **safe** if all the **reachable** states satisfy the property $\neg \text{Bad}$

Inductive invariants



System S is **safe** if all the **reachable** states satisfy the property $\neg \text{Bad}$

System S is safe iff there exists an **inductive invariant** *Inv* :

Init $\subseteq$ *Inv* (**Initiation**)

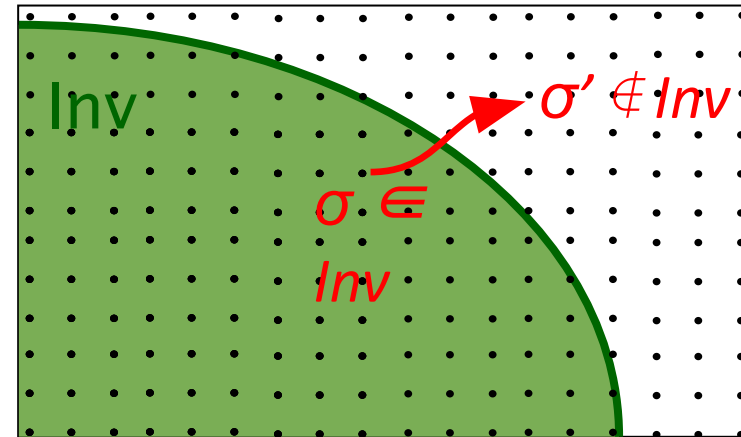
if $\sigma \in \text{Inv}$ and $\sigma \rightarrow \sigma'$ then $\sigma' \in \text{Inv}$ (**Consecution**)

Inv $\cap$ *Bad* = $\emptyset$ (**Safety**)

} translated to VCs

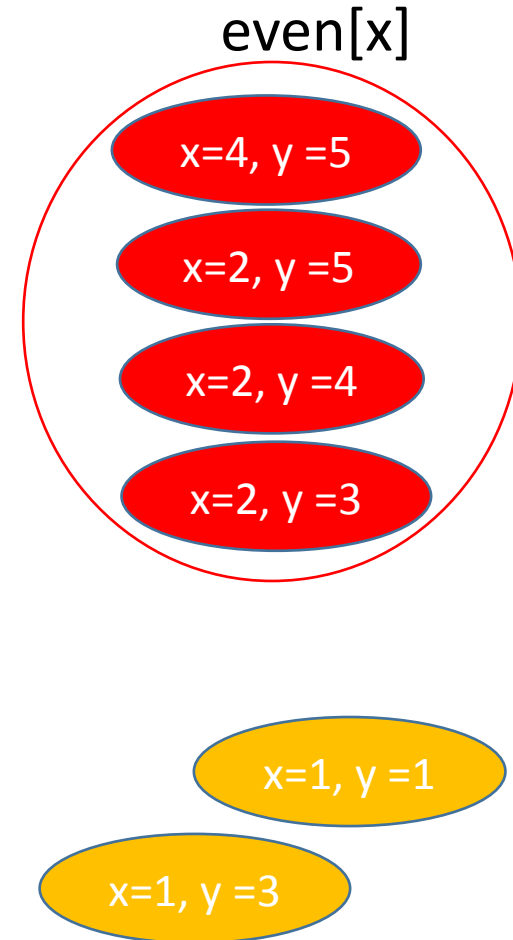
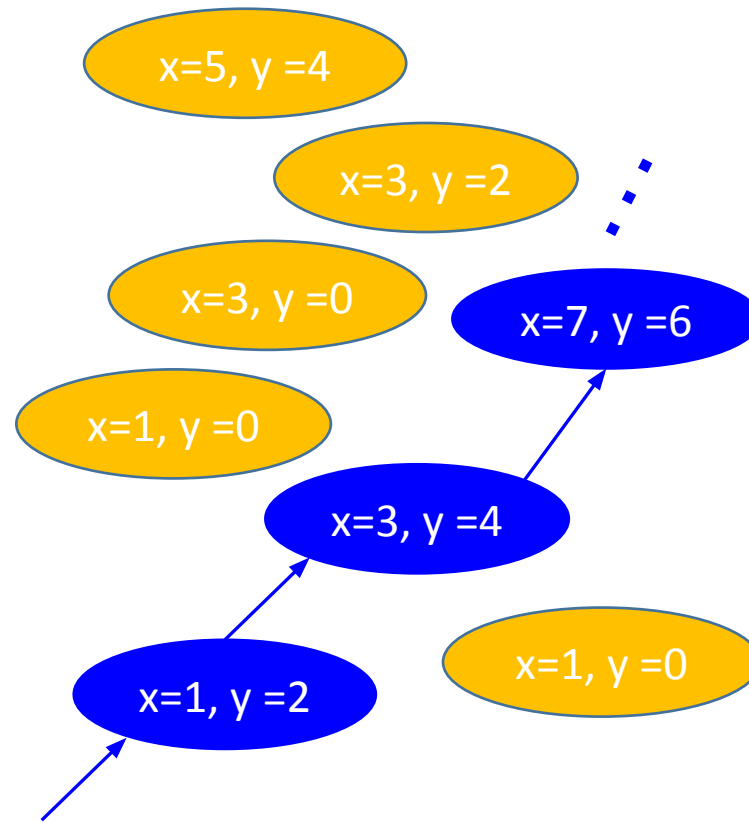
Counterexample To Induction (CTI)

- States σ, σ' are a CTI of Inv if:
 - $\sigma \in Inv$
 - $\sigma' \notin Inv$
 - $\sigma \sqsubseteq \sigma'$
- A CTI may indicate:
 - A bug in the system
 - A bug in the safety property
 - A bug in the inductive invariant
 - Too weak
 - Too strong



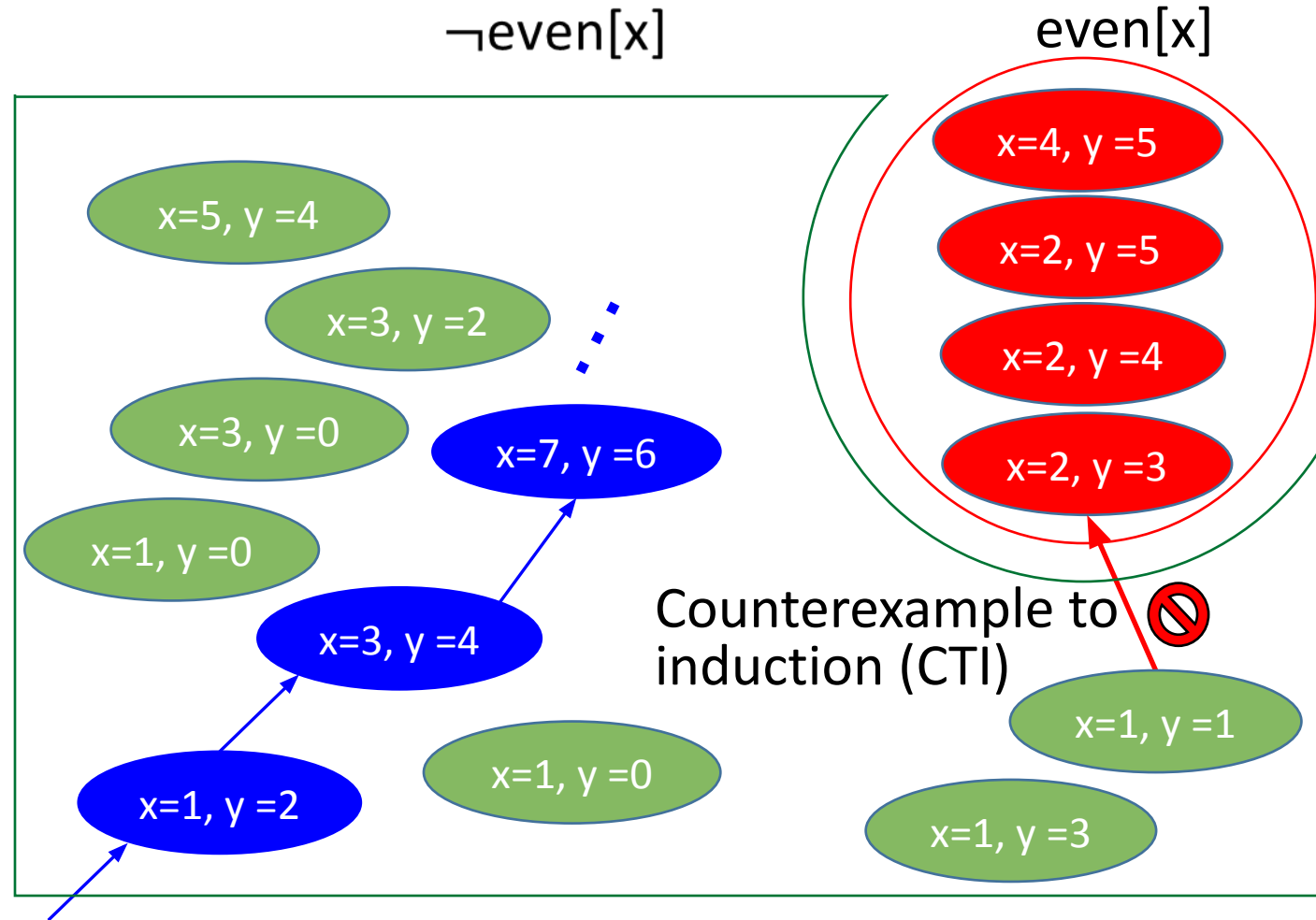
Simple example: loop invariants

```
x := 1;  
y := 2;  
while * do {  
  assert  $\neg \text{even}[x]$ ;  
  TR | x := x + y;  
      | y := y + 2;  
}
```



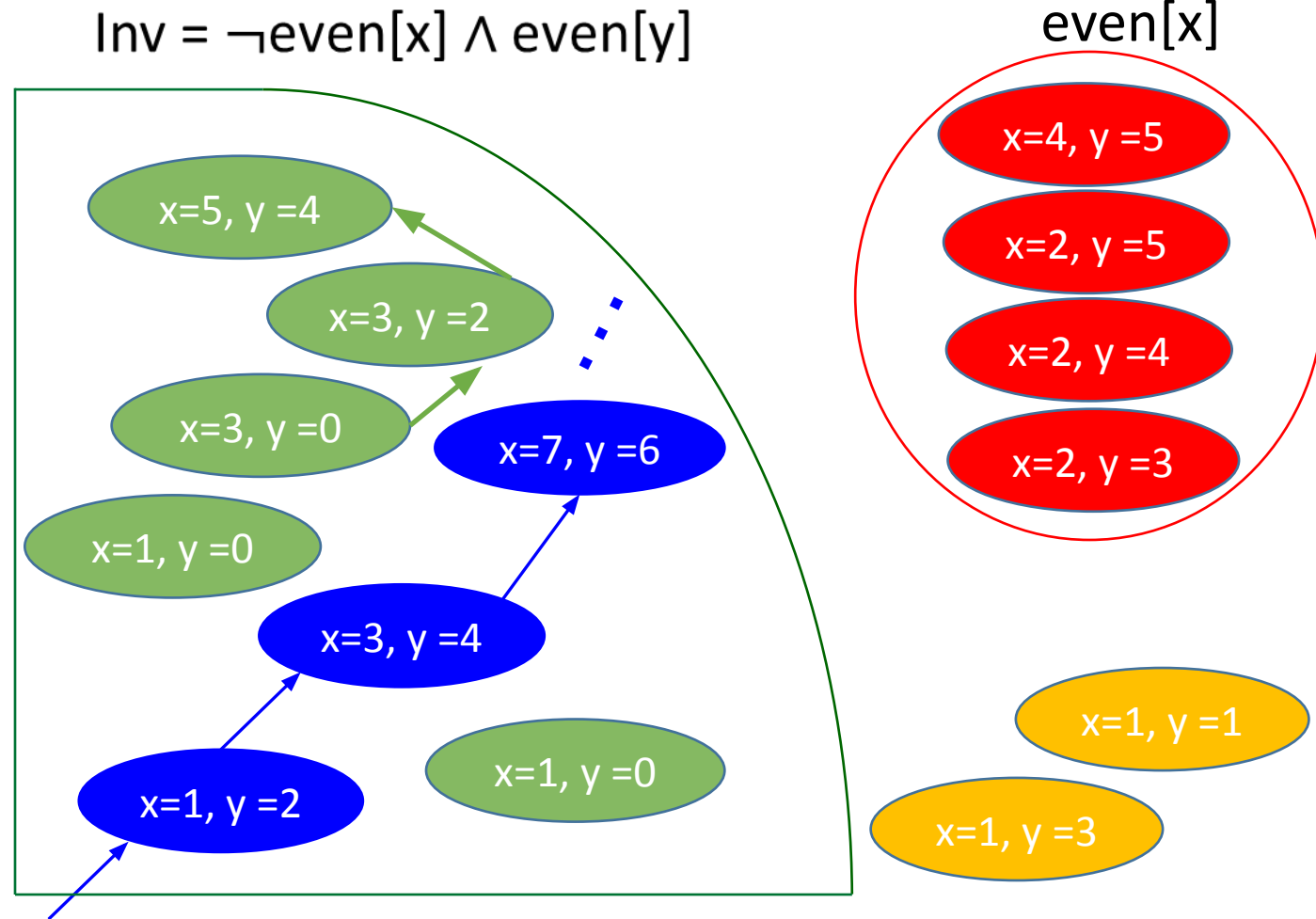
Simple example: loop invariants

```
x := 1;  
y := 2;  
while * do {  
  assert  $\neg\text{even}[x]$ ;  
  TR | x := x + y;  
      y := y + 2;  
}
```



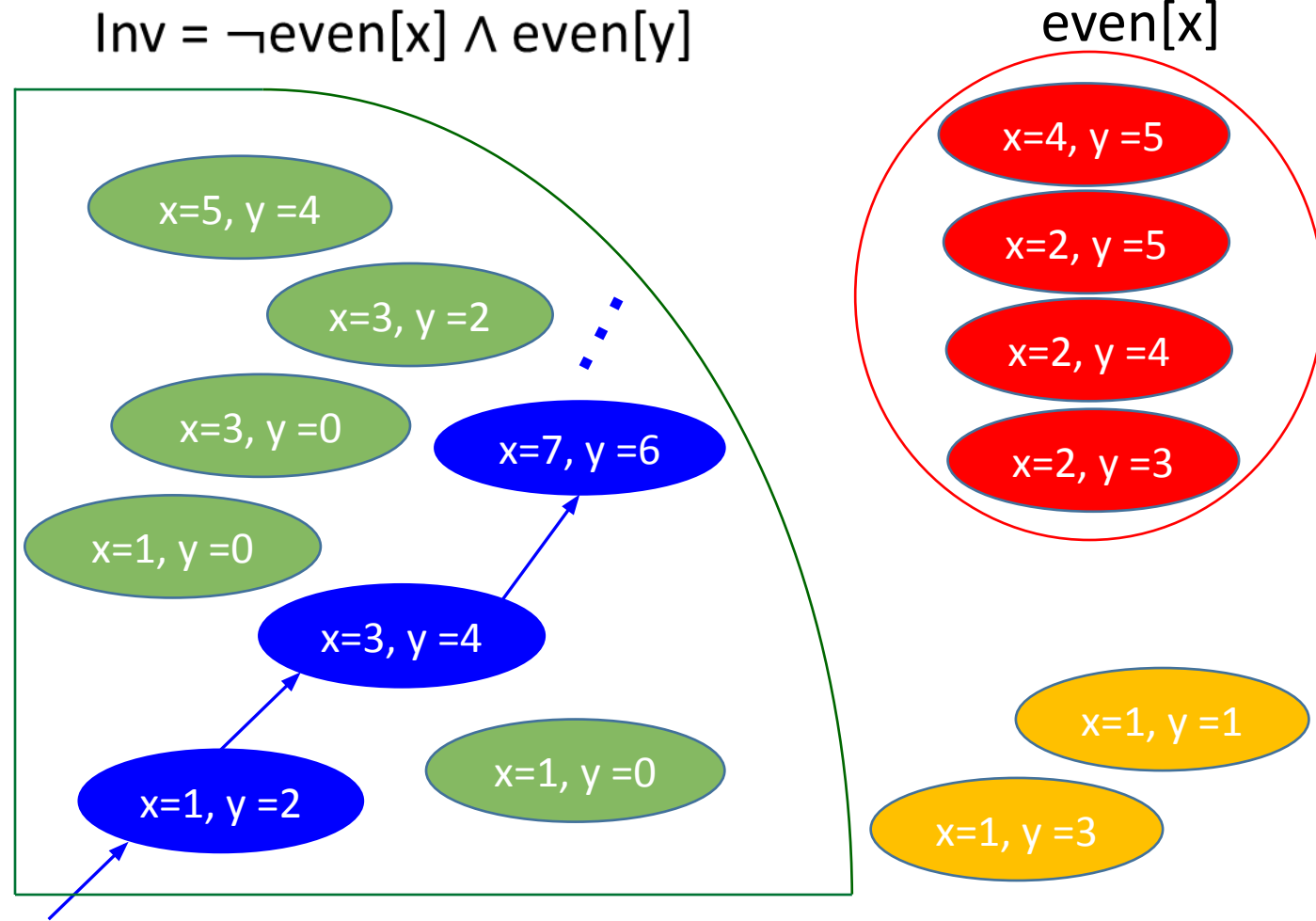
Simple example: loop invariants

```
x := 1;  
y := 2;  
while * do {  
  assert  $\neg\text{even}[x]$ ;  
  TR | x := x + y;  
      | y := y + 2;  
}
```



Simple example: loop invariants

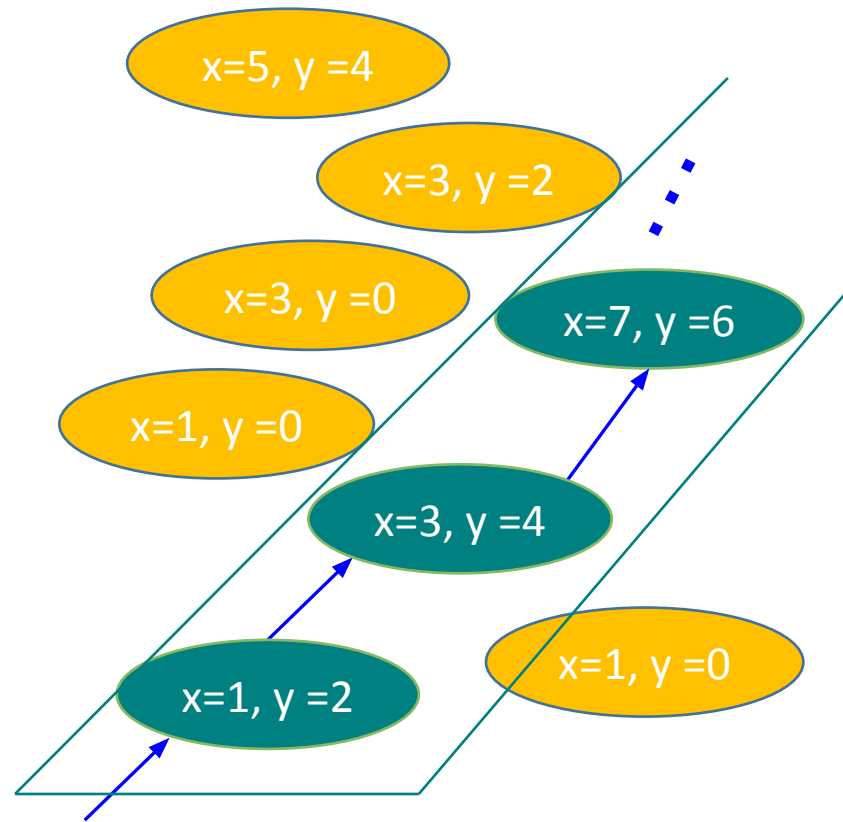
```
x := 1;  
y := 2;  
while * do {  
  assert  $\neg\text{even}[x]$ ;  
  TR |  $x := (x*x - y*y)/(x - y);$   
  |  $y := y + 2;$   
}
```



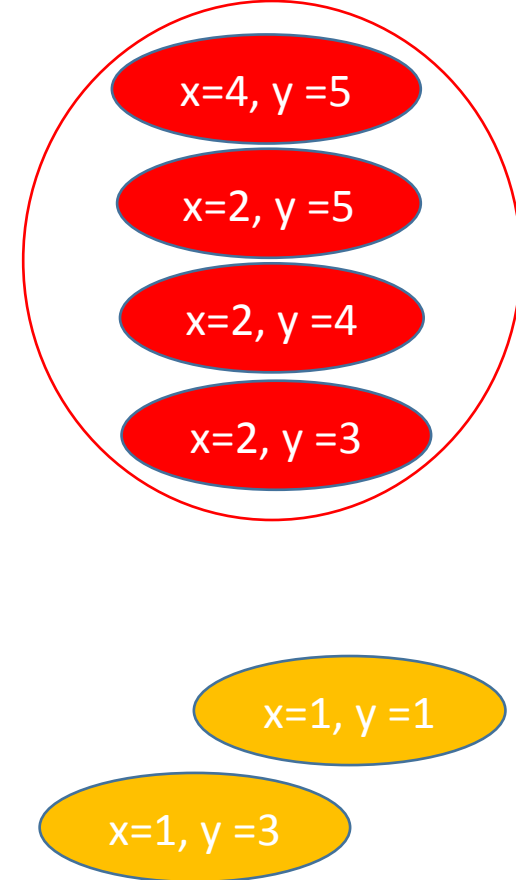
Simple example: loop invariants

$$\text{Inv} = y^2 - 2y - 4x + 4 = 0$$

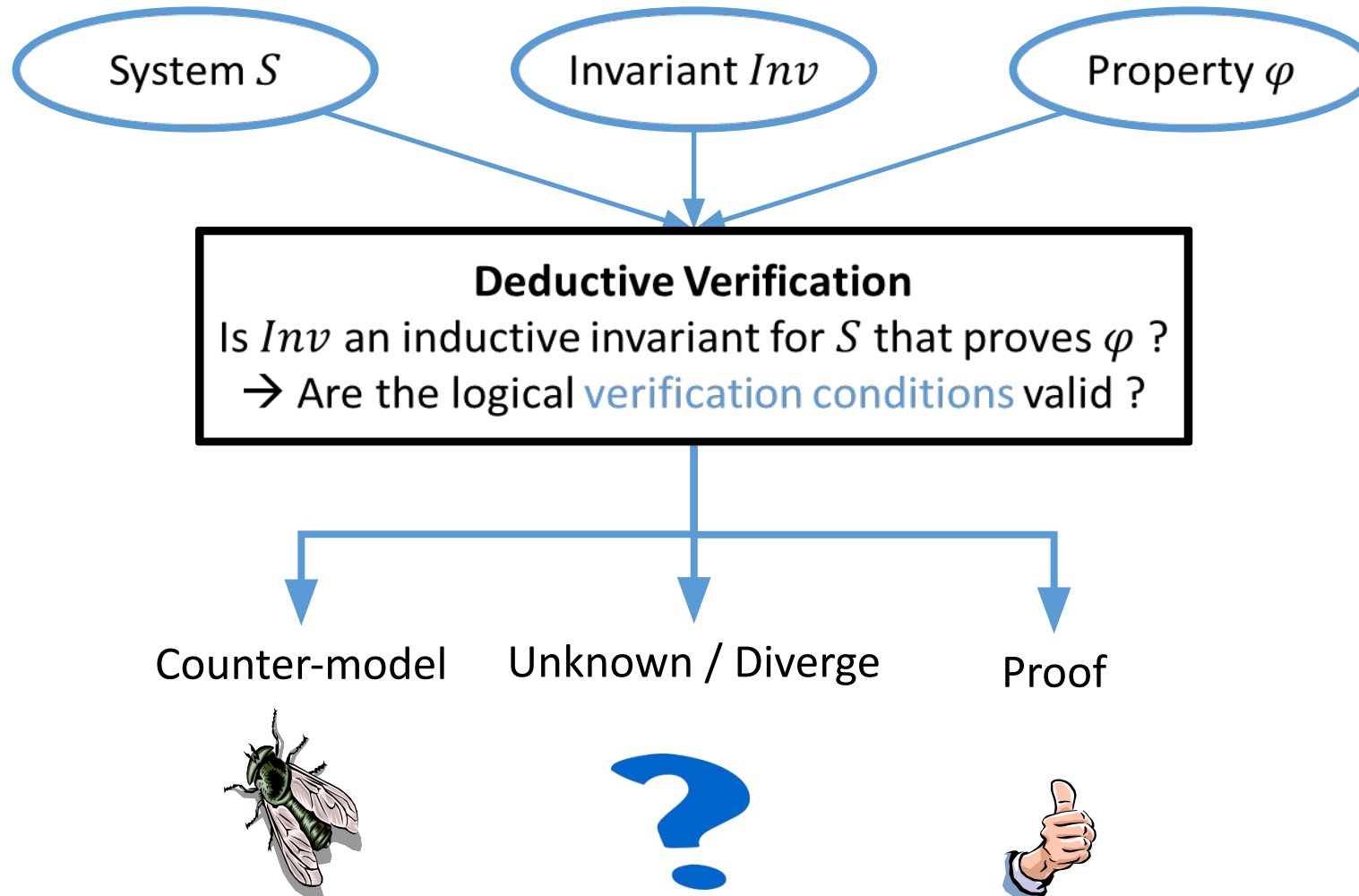
```
x := 1;  
y := 2;  
while * do {  
  assert ¬even[x];  
  TR | x := x + y;  
      y := y + 2;  
}
```



even[x]



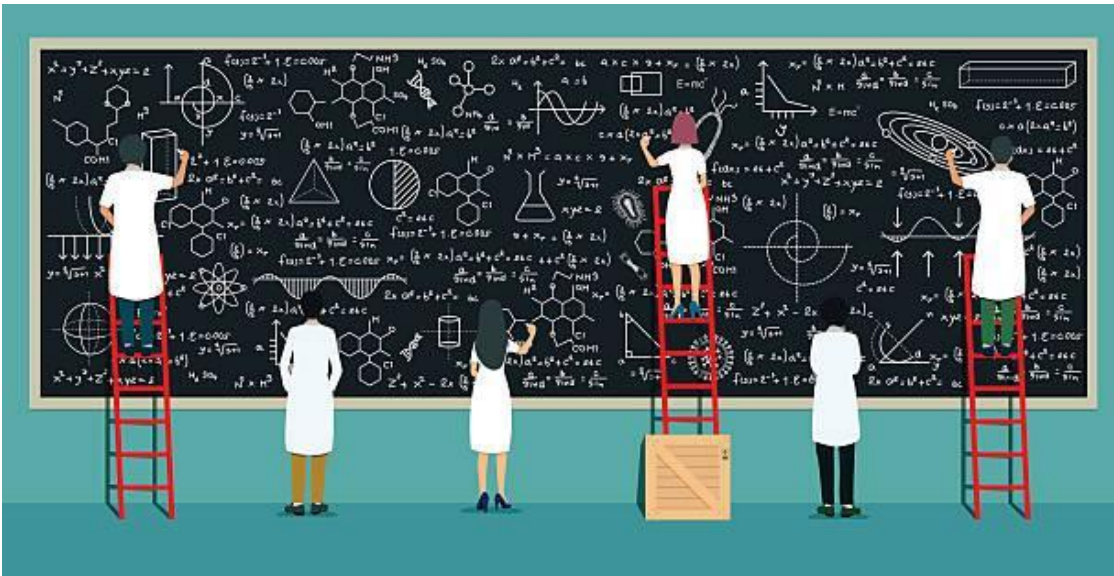
Deductive verification



Verifying distributed systems is hard

Verdi

Verification of Raft in Coq
50,000 lines of manual proof



IronFleet

Verification of Multi-Paxos in Dafny
12,000 lines and 3.7 person-years
Uses solver for **undecidable** SMT checks

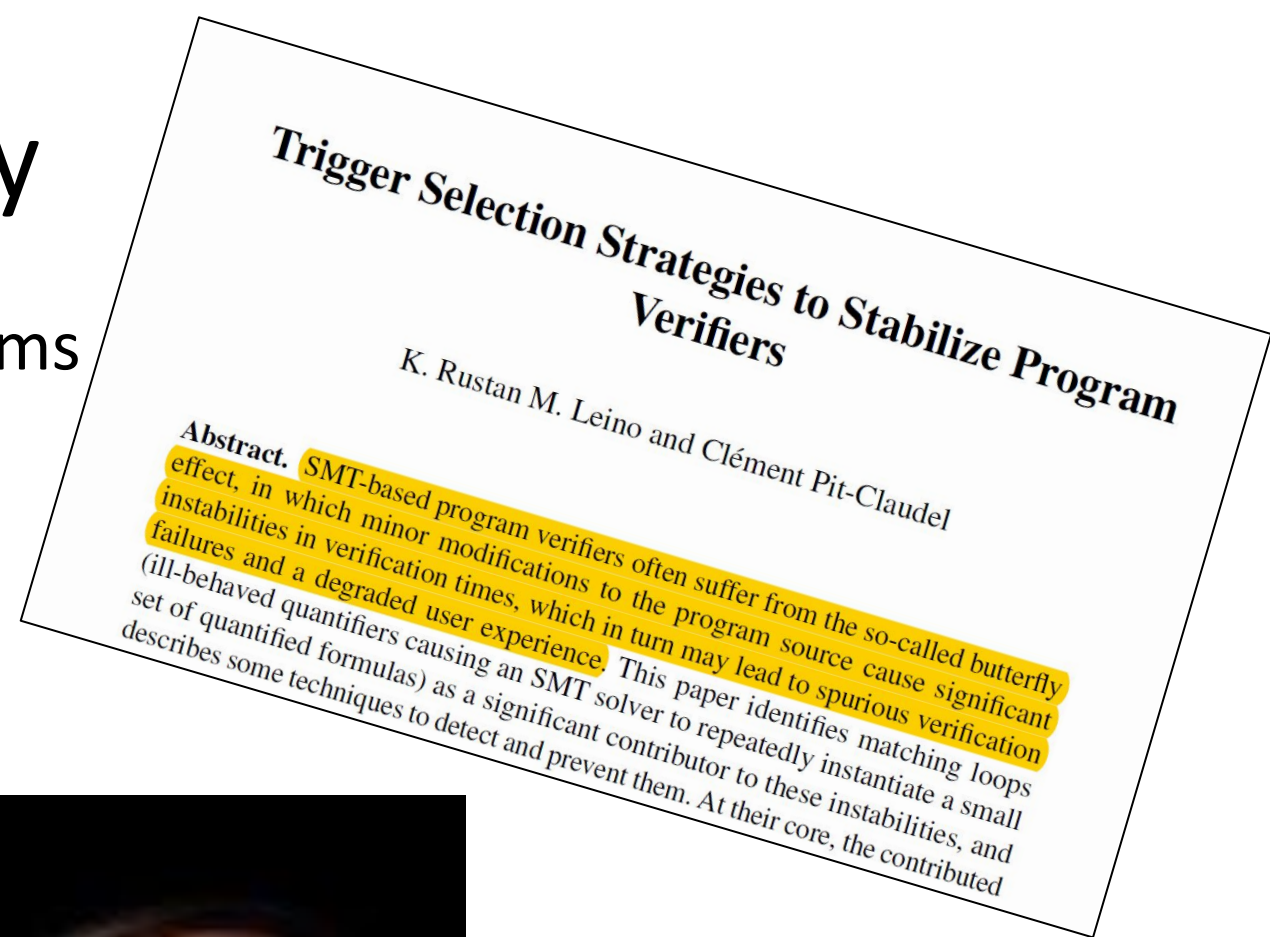


[SOSP'15] Hawblitzel et al. IronFleet: proving practical distributed systems correct

[PLDI'15] Wilcox et al. Verdi: a framework for implementing and formally verifying distributed systems

Effects of undecidability

- The verifier may fail on tiny programs
- No explanation when tactics fails
 - Counterproofs
- The butterfly effect



The challenge of automated deduction

from: Ferraiulo, Baumann, Hawblitzel, Parno, “Komodo: Using Verification to Disentangle Secure-enclave Hardware from Software”, SOSPP ’17

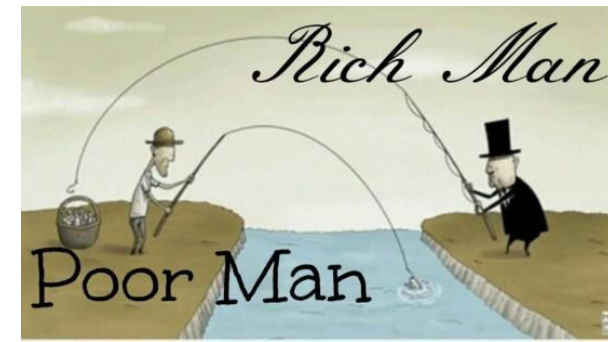
“ The most frustrating recurring problem was proof instability [...] Timeouts are challenging to debug, because the solver generally fails to provide useful feedback [...] even once fixed, the proof may easily timeout again due to minor perturbations. Worse, minor changes can trigger timeouts in seemingly unrelated proofs ”

Gap: deductive power of automated provers is not translating into verification productivity

Rich logic, Poor logic

Rich SMT Theories

- Linear arithmetic, Bitvectors, Arrays, Strings, Datatypes, ...
- Great tools: Yices, Z3, CVC4, Boolector, MathSAT, SMTInterpol, ...
- Essential in Dafny, Sage, Klee, F*,
- Hides complexity from the user
- But solvers are heuristic, unpredictable, and not transparent

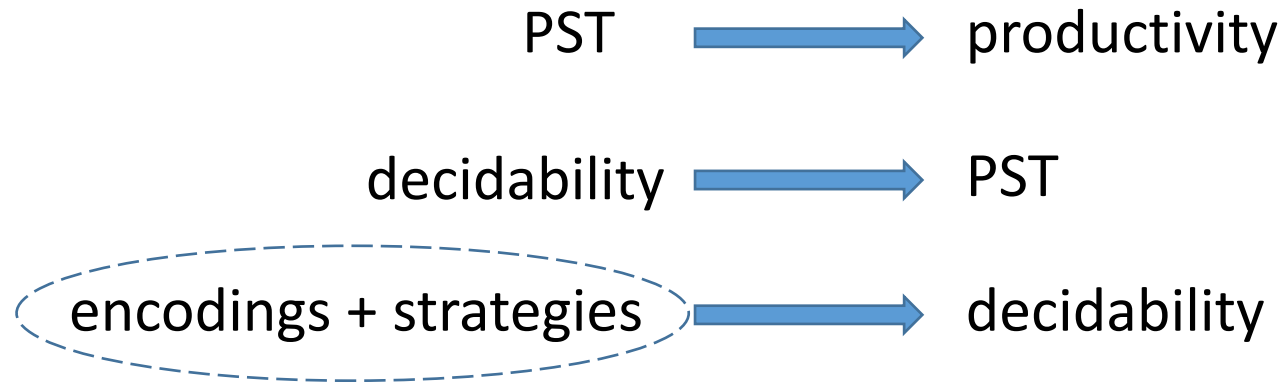


The Ivy deductive verification system – design hypotheses

PST

- **Predictability**: we can reasonably predict solver performance
- **Stability (or continuity)**: small, irrelevant changes do not greatly affect performance
- **Transparency**: failures have understandable explanations

Hypotheses:

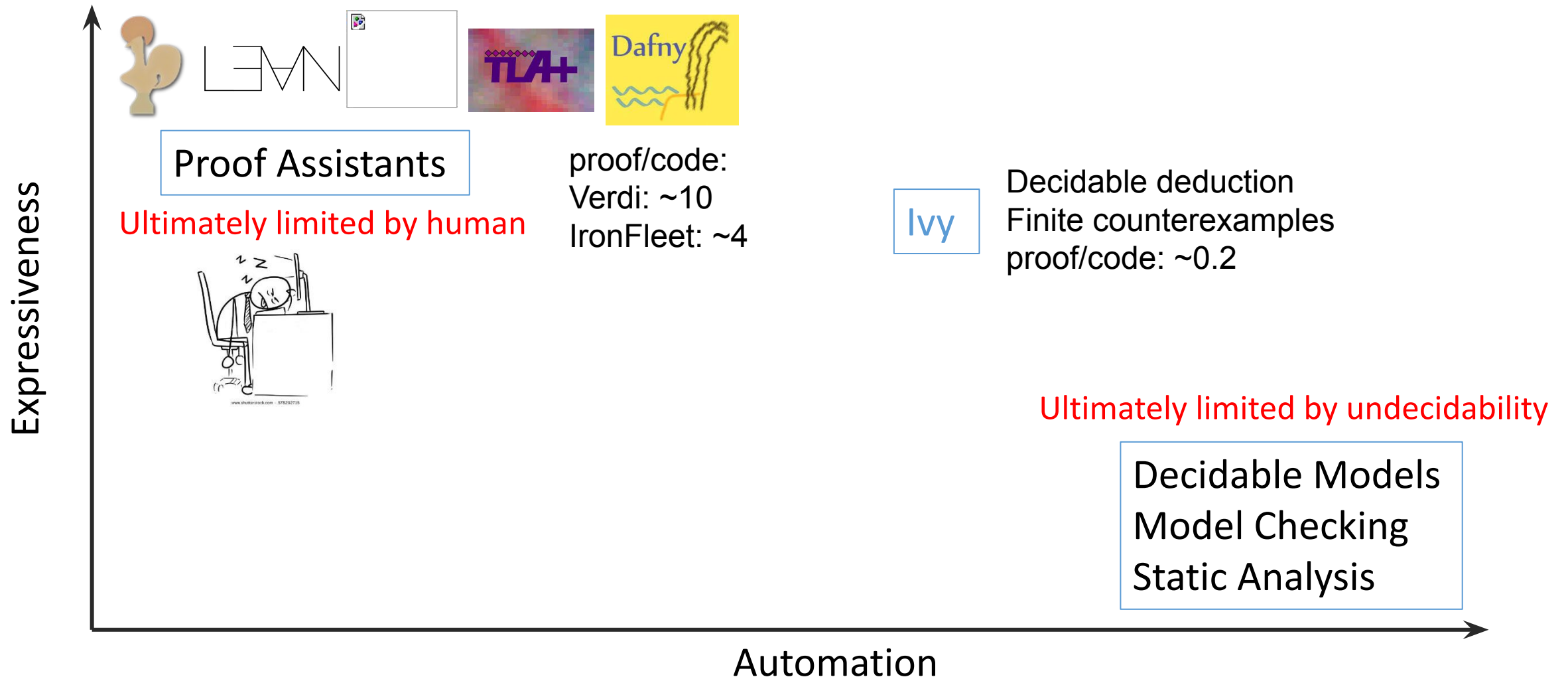


[PLDI'16] O.P., Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, Sharon Shoham.

Ivy: Safety Verification by Interactive Generalization

[CAV'20] O.P., Kenneth L. McMillan. Ivy: A Multi-modal Verification Tool for Distributed Algorithms

Design space for formal verification



Ivy's principles

- Specify systems and properties in decidable fragment of first-order logic
 - Allows quantifiers to reason about unbounded sets
 - Decidable to check inductiveness
 - Finite counterexamples to induction, display graphically or textually
- Interact with the user to find inductive invariants
- Challenge: use restricted logic to verify interesting systems
 - Transitive closure: network topology
 - Sets and cardinalities: Paxos, Byzantine Fault Tolerance, Reconfiguration
 - Liveness and temporal properties

Effectively Propositional Logic – EPR

a.k.a. Bernays-Schönfinkel-Ramsey class

- Limited fragment of first-order logic **without theories**

- Relations, but no functions

$r(a, b)$ ✓

- Restricted quantifier prefix: $\exists^* \forall^* \varphi_{QF}$

$\exists x \forall y. r(x, y)$ ✓

$r(a, f(a))$

- Finite model property

- A formula is satisfiable iff it has a model of size $\leq$
 $\# \text{ constant symbols} + \# \text{ existential variables}$

$\forall x \exists y. r(x, y)$

- Complexity:

- NEXPTIME-complete
- Σ_2^P if relation arities are bounded by a constant
- NP if quantifier prefix is also bounded by a constant



EPR Satisfiability



Skolem

$$\exists x, y. \forall z. r(x, z) \leftrightarrow r(z, y)$$

$$=_{\text{SAT}} \forall z. r(c_1, z) \leftrightarrow r(z, c_2)$$



Herbrand

$$=_{\text{SAT}} (r(c_1, c_1) \leftrightarrow r(c_1, c_2)) \wedge (r(c_1, c_2) \leftrightarrow r(c_2, c_2))$$

$$=_{\text{SAT}} (p_{11} \leftrightarrow p_{12}) \wedge (p_{12} \leftrightarrow p_{22})$$

Effectively Propositional Logic – EPR a.k.a. Bernays-Schönfinkel-Ramsey class

- Limited fragment of first-order logic **without theories**
 - Relations, but no functions
 - Restricted quantifier prefix: $\exists^* \forall^* \varphi_{QF}$
- Finite model property
 - A formula is satisfiable iff it has a model of size:
constant symbols + # existential variables
- Complexity:
 - NEXPTIME-complete
 - Σ_2^P if relation arities are bounded by a constant
 - NP if quantifier prefix is also bounded by a constant



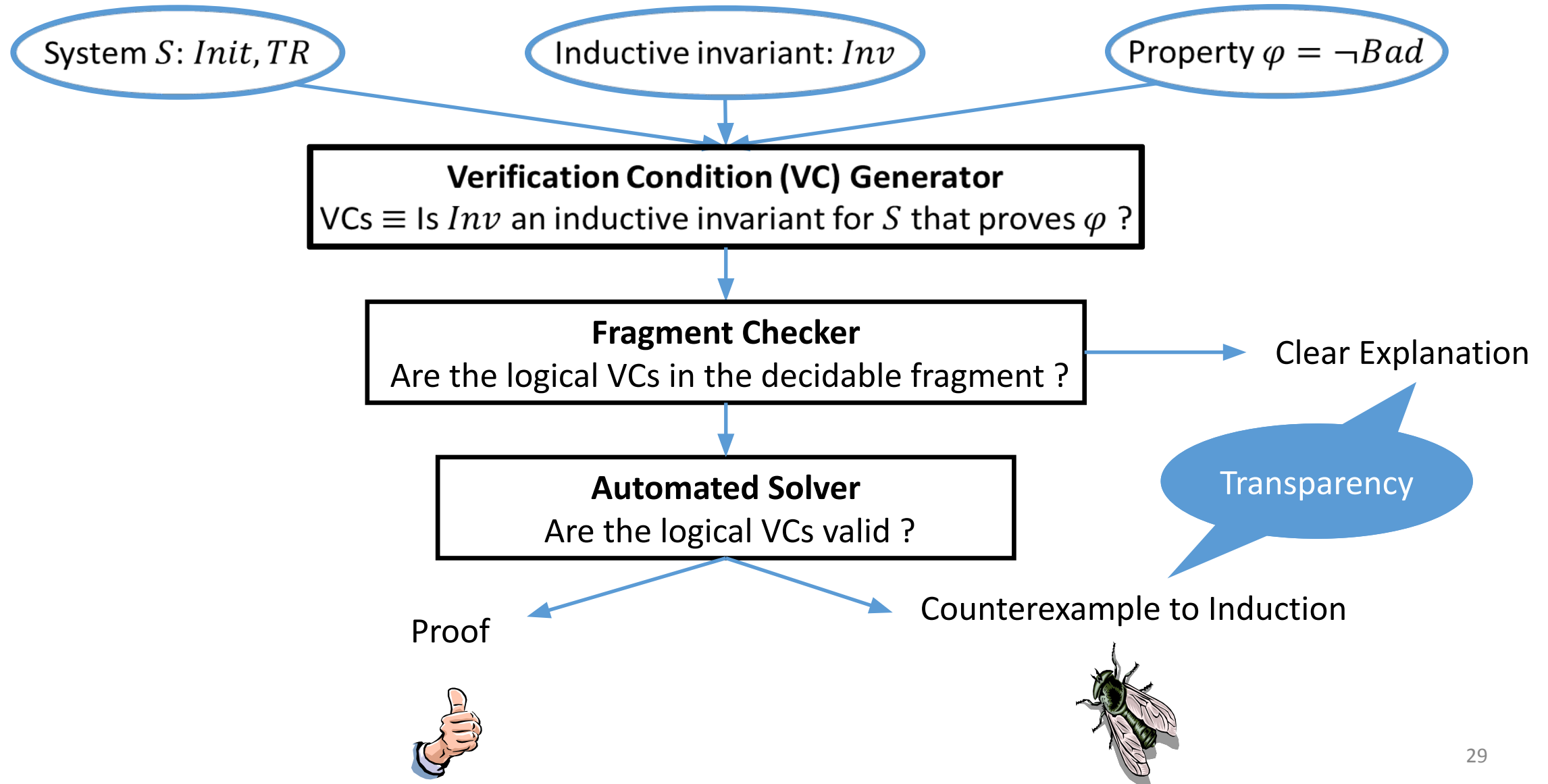
Decidable Fragments in Ivy

- Many-sorted EPR: allows **acyclic** functions and quantifier alternations
 - E.g., $f: A \rightarrow B$ but not $g: B \rightarrow A$
 - Maintains finite model property
 - Finite complete instantiations, supported by Z3
-
- QFLIA – Quantifier Free Linear Integer Arithmetic
 - FAU – Finite Almost Uninterpreted [CAV'07]
 - Allow limited arithmetic + acyclic quantifier alternations
 - Maintains finite complete instantiations

Protocol
Designs

System
Implementations

Logic-based deductive verification in Ivy

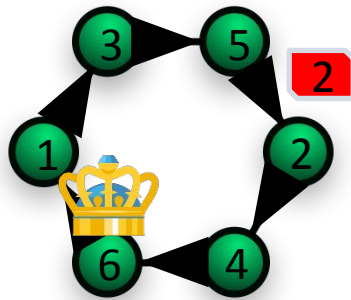
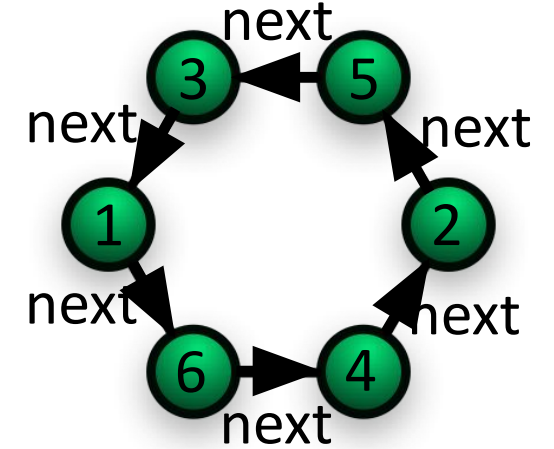


Challenge: encoding into EPR

- Many-sorted EPR: allows **acyclic** functions and quantifier alternations
 - E.g., $f: A \rightarrow B$ without $g: B \rightarrow A$
 - Maintains small model property of EPR
 - Finite complete instantiations
- But what can you possibly express in such a restricted logic?
 - Transitive closure over deterministic paths
 - Set cardinalities
 - Avoiding quantifier alternations
 - Encoding liveness and LTL

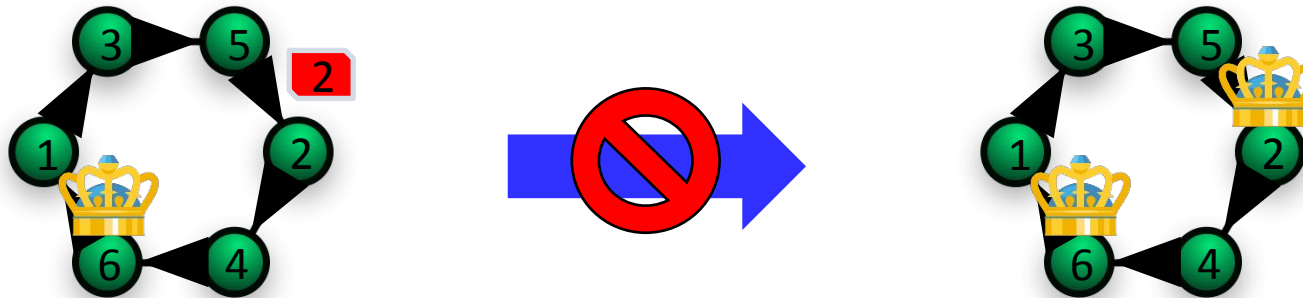
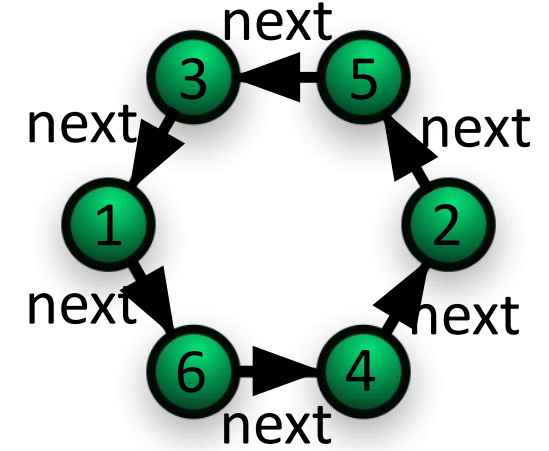
Example: Leader election in a ring

- Unidirectional ring of nodes, unique numeric ids
- Protocol:
 - Each node sends its id to the next
 - Upon receiving a message, a node passes it (to the next) if the id in the message is higher than the node's own id
 - A node that receives its own id becomes a leader
- Theorem: at any given time there is at most one leader
 - Inductive?



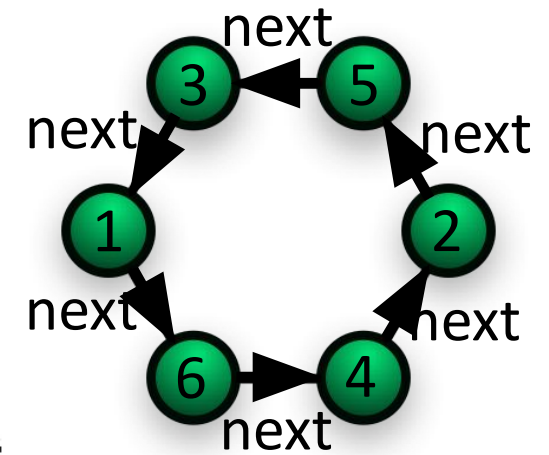
Example: Leader election in a ring

- Unidirectional ring of nodes, unique numeric ids
- Protocol:
 - Each node sends its id to the next
 - Upon receiving a message, a node passes it (to the next) if the id in the message is higher than the node's own id
 - A node that receives its own id becomes a leader
- Theorem: at any given time there is at most one leader
 - Inductive? **NO**



Example: Leader election in a ring

- Unidirectional ring of nodes, unique numeric ids
- Protocol:
 - Each node sends its id to the next
 - Upon receiving a message, a node passes it (to the next) if the id is higher than its own
 - A node is elected leader if it receives its own message
- Theorem
 - Proposition:** This algorithm detects one and only one highest number.
 - Argument:** By the circular nature of the configuration and the consistent direction of messages, any message must meet all other processes before it comes back to its initiator. Only one message, that with the highest number, will not encounter a higher number on its way around. Thus, the only process getting its own message back is the one with the highest number.

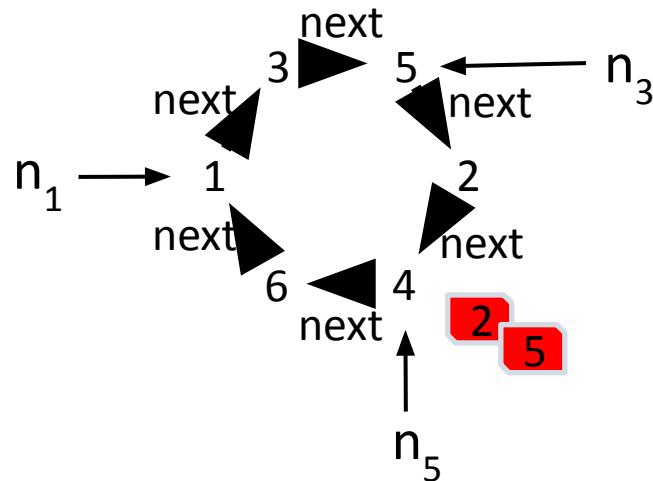


Leader election protocol – first-order logic

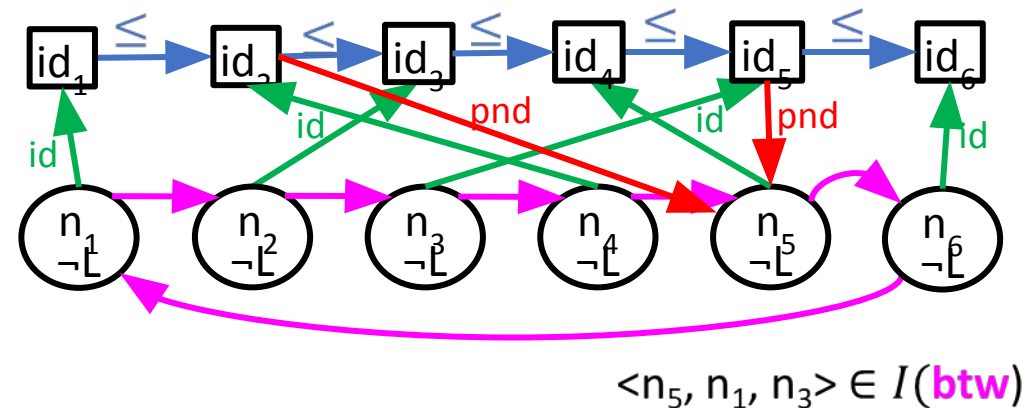
- $\leq$ (ID, ID) – total order on node id's
- **btw** (Node, Node, Node) – the ring topology
- **id**: Node $\square$ ID – relate a node to its unique id
- **pending**(ID, Node) – pending messages
- **leader**(Node) – leader(n) means n is the leader

} Axiomatized in first-order logic

protocol state



first-order structure



Leader election protocol – first-order logic

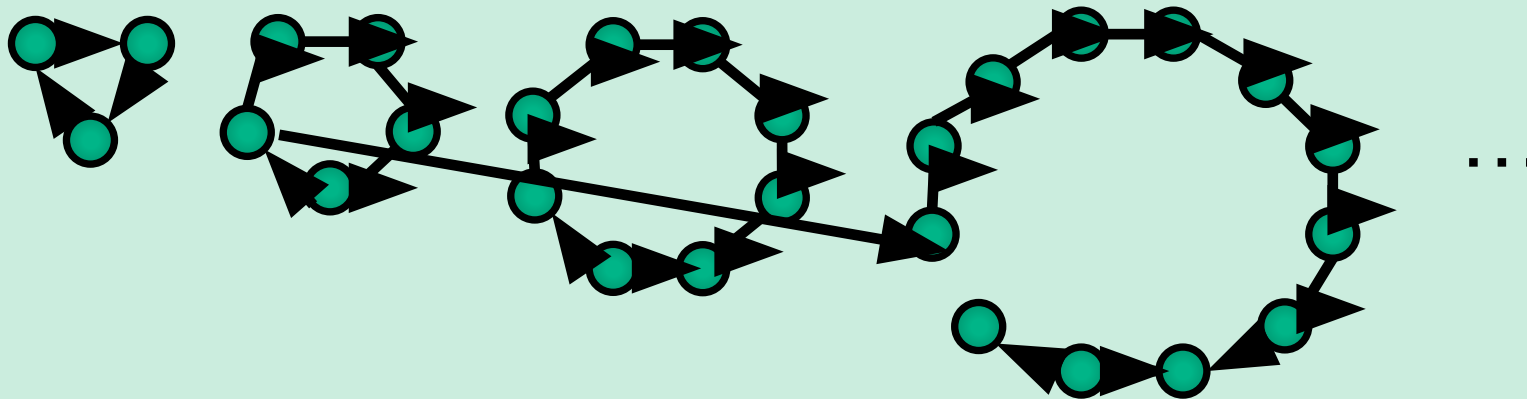
- $\leq$ (ID, ID) – total order on node id's
- **btw** (Node, Node, Node) – the ring topology
- **id**: Node $\square$ ID – relate a node to its unique id
- **pending**(ID, Node) – pending messages
- **leader**(Node) – leader(n) means n is the leader

} Axiomatized in first-order logic

protocol state

first-order structure

Specify and verify the protocol for **any** number of nodes in the ring



Leader election protocol – first-order logic

- $\leq$ (ID, ID) – total order on node id's
- **btw** (Node, Node, Node) – the ring topology
- **id**: Node $\square$ ID – relate a node to its unique id
- **pending**(ID, Node) – pending messages
- **leader**(Node) – leader(n) means n is the leader

```
action send(n: Node)
  "s := next(n)"
  pending(id(n), s) := true
```

```
action receive(n: Node, m: ID)
  requires pending(m, n)
  if * then
    pending(m, n) := false
  if id(n) = m then
    leader(n) := true
  else if id(n)  $\leq$  m then
    "s := next(n)"
    pending(m, s) := true
```

TR:

$\exists n, s: \text{Node}. "s = \text{next}(n)" \wedge \forall x: \text{ID}, y: \text{Node}. \text{pending}'(x, y) \leftrightarrow (\text{pending}(x, y) \vee (x = \text{id}(n) \wedge y = s))$

Bad:

$\text{assert } I0 = \forall x, y: \text{Node}. \text{leader}(x) \wedge \text{leader}(y) \rightarrow x = y$

Leader election protocol – inductive invariant

Safety property: I_0

$$I_0 = \forall x, y: \text{Node}. \text{leader}(x) \wedge \text{leader}(y) \rightarrow x = y$$

Inductive invariant: $\text{Inv} = I_0 \wedge I_1 \wedge I_2 \wedge I_3$

$$I_1 = \forall n_1, n_2: \text{Node}. \text{leader}(n_2) \rightarrow \text{id}[n_1] \leq \text{id}[n_2]$$

The leader has the highest ID

$$I_2 = \forall n_1, n_2: \text{Node}. \text{pending}(\text{id}[n_2], n_2) \rightarrow \text{id}[n_1] \leq \text{id}[n_2]$$

Only the leader can be self-pending

$$I_3 = \forall n_1, n_2, n_3: \text{Node}. \text{btw}(n_1, n_2, n_3) \wedge \text{pending}(\text{id}[n_2], n_1) \rightarrow \text{id}[n_3] \leq \text{id}[n_2]$$

Cannot bypass higher nodes

- $\leq$ (ID, ID) – total order on node id's
- **btw** (Node, Node, Node) – the ring topology
- **id**: Node $\square$ ID – relate a node to its unique id
- **pending**(ID, Node) – pending messages
- **leader**(Node) – leader(n) means n is the leader

Leader election protocol – inductive invariant

Safety property: I_0

$$I_0 = \forall x, y: \text{Node}. \text{leader}(x) \wedge \text{leader}(y) \rightarrow x = y$$

Inductive invariant: $\text{Inv} = I_0 \wedge I_1 \wedge I_2 \wedge I_3$

$$I_1 = \forall n_1, n_2: \text{Node}. \text{leader}(n_2) \rightarrow \text{id}[n_1] \leq \text{id}[n_2]$$

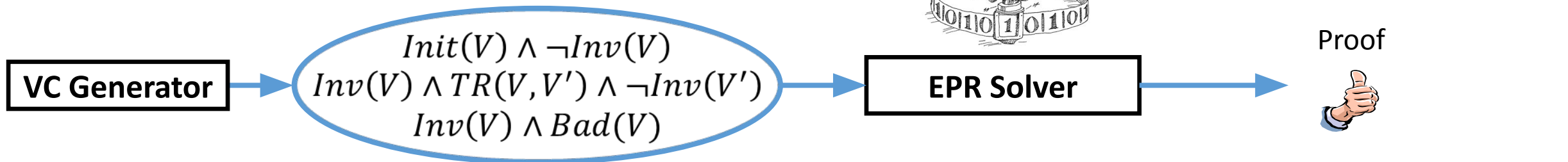
The leader has the highest ID

$$I_2 = \forall n_1, n_2: \text{Node}. \text{pending}(\text{id}[n_2], n_2) \rightarrow \text{id}[n_1] \leq \text{id}[n_2]$$

Only the leader can be self-pending

$$I_3 = \forall n_1, n_2, n_3: \text{Node}. \text{btw}(n_1, n_2, n_3) \wedge \text{pending}(\text{id}[n_2], n_1) \rightarrow \text{id}[n_1] \leq \text{id}[n_2]$$

I can decide EPR!
cannot bypass higher nodes

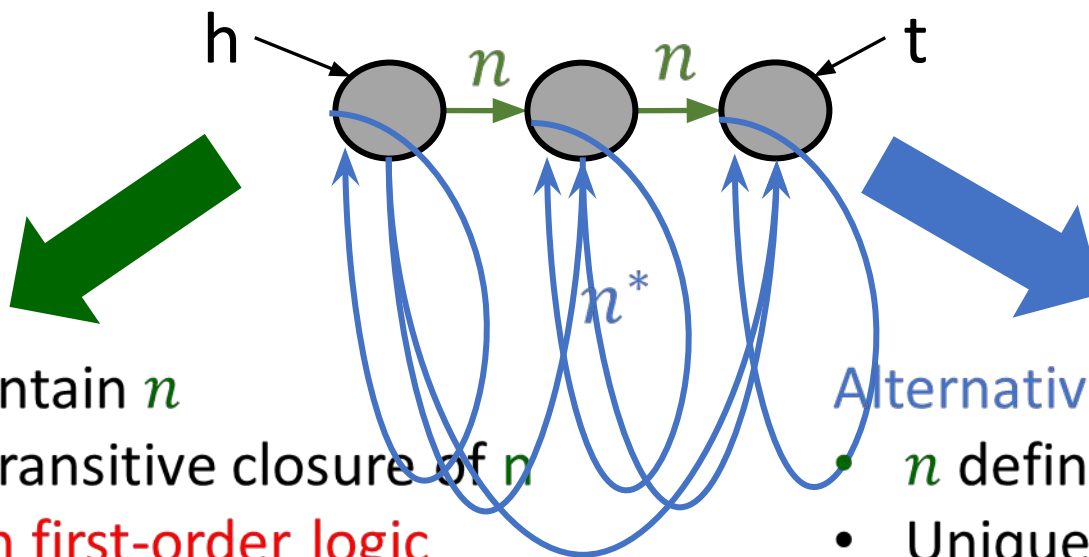


Principle: first-order abstractions

Concept	Intention	First-order abstraction
Node ID's	Integers	
Ring Topology	Next edges + Transitive closure	

Key idea: representing deterministic paths

[Shachar Itzhaky PhD, SIGPLAN Dissertation Award 2016]



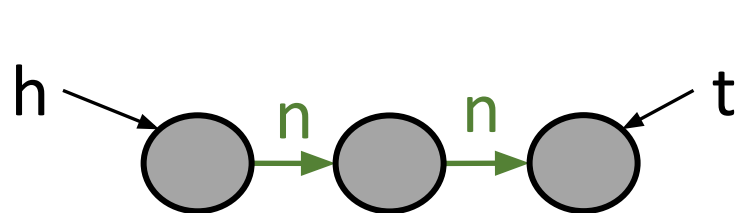
Alternative 1: maintain n

- n^* defined by transitive closure of n
- not definable in first-order logic

Alternative 2: maintain n^*

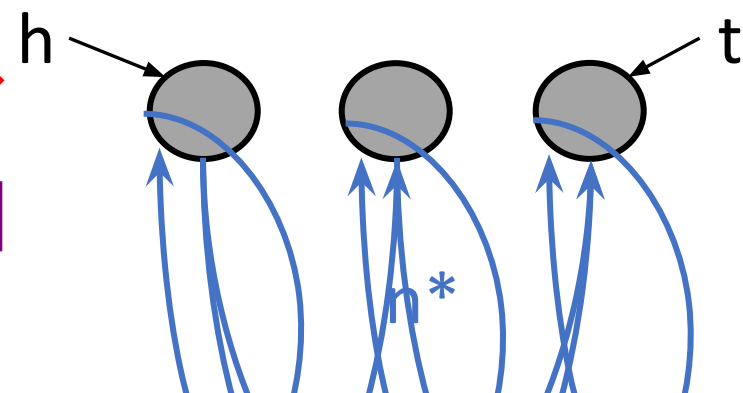
- n defined by transitive reduction of n^*
- Unique due to outdegree ≤ 1
- Definable in first order logic

$$n(x, y) \equiv n^*(x, y) \wedge x \neq y \wedge \forall z. n^*(x, z) \rightarrow z = y \vee z = x$$

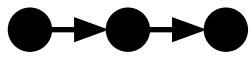


Not first order expressible

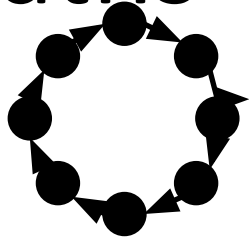
First order expressible



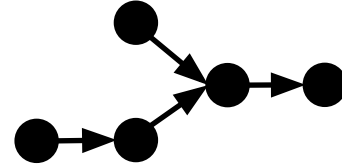
Sound and complete* axiomatization of deterministic paths



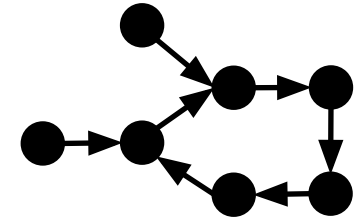
Line
 $\leq (x, y)$



Ring
 $btw(x, y, z)$



Forest, Tree,
Acyclic partial function
 $\leq (x, y)$



Graph with out degree 1,
General partial function
 $p(x, y, z)$

For every class C of finite graphs above:

- Axioms for path relation – universally quantified
- Successor formula – 1 universal quantifier
- Update formulas for node / edge addition and removal – **Dyn-EPR**
- Soundness Theorem *Every graph of class C satisfies the axioms of C*
Edges agree with successor formula
- Completeness Theorem *Every finite structure satisfying the axioms of C is isomorphic (paths and edges) to a graph of class C*

Sound and complete* axiomatization of deterministic paths



Line

$\leq (x, y)$

Ring

$btw(x, y, z)$

Ac

For every class C of finite graphs above:

- Axioms for path relation – universally quantified
- Successor formula – 1 universal quantifier
- Update formulas for node / edge addition and removal – **Dyn-EPR**
- **Soundness Theorem** *Every graph of class C satisfies the axioms of C*
Edges agree with successor formula
- **Completeness Theorem** *Every finite structure satisfying the axioms of C is isomorphic (paths and edges) to a graph of class C*

Universally quantified formulas
□ finite model property
+ Completeness Thm. for finite structures

Sound and complete automatic
deductive verification

Challenge: encoding into EPR

- But what can you possibly express in such a restricted logic?
 - Transitive closure over deterministic paths
 - Set cardinalities
 - Avoiding quantifier alternations
 - Encoding liveness and LTL

Paxos

- Family of distributed consensus protocols
 - let nodes maintain consistent state under crashes and packet loss
- Variants for different tradeoffs and extra features
- Active research and extensive industry use
 - Pervasive approach to fault-tolerant distributed computing



Sets and cardinalities in EPR

- Consensus algorithms use set cardinalities
 - Wait for messages from **more than $N / 2$ nodes**
- Set Cardinalities + Arithmetic + Uninterpreted **$\gg$ EPR?**
- **Insight: set cardinalities are used to get a simple effect**
Can be modeled in first-order logic and EPR!
- Axiomatize quorums in first-order logic:
sort Quorum
relation $m(\text{Node}, \text{Quorum})$
 - set membership (2nd-order logic in first-order)**axiom** $\forall q_1, q_2: \text{Quorum}. \exists n: \text{Node}. m(n, q_1) \wedge m(n, q_2)$

```
action propose(r:Round) {  
  require  $|\{n \mid \varphi(n)\}| > \frac{N}{2}$   
  ...  
}
```



```
action propose(r:Round) {  
  require  $\exists q. \forall n. m(n, q) \rightarrow \varphi(n)$   
  ...  
}
```

✓ EPR

Principle: first-order abstractions

Concept	Intention	First-order abstraction
Messages	Network semantics: dropping duplication reordering	relation <code>start_msg</code> (Round) relation <code>join_msg</code> (Node, Round, Round, Value) relation <code>propose_msg</code> (Round, Value) relation <code>vote_msg</code> (Node, Round, Value)
Quorums	Majority sets	
Byzantine Quorums	2/3 Majority	

Paxos in first-order logic

```

1 sort node, quorum, round, value
2
3 relation ≤ : round, round
4 axiom total_order(≤)
5 constant ⊥ : round
6
7 relation member : node, quorum
8 axiom ∀ q1, q2 : quorum. ∃ n : node. member(n, q1) ∧ member(n, q2)
9
10 relation start_round_msg : round
11 relation join_ack_msg : node, round, round, value
12 relation propose_msg : round, value
13 relation vote_msg : node, round, value
14 relation decision : node, round, value
15
16 init ∀ r. ¬start_round_msg(r)
17 init ∀ n, r1, r2, v. ¬join_ack_msg(n, r1, r2, v)
18 init ∀ r, v. ¬propose_msg(r, v)
19 init ∀ n, r, v. ¬vote_msg(n, r, v)
20 init ∀ n, r, v. ¬decision(n, r, v)

```

```

21
22 action START_ROUND(r : round) {
23   assume r ≠ ⊥
24   start_round_msg(r) := true
25 }
26 action JOIN_ROUND(n : node, r : round) {
27   assume r ≠ ⊥
28   assume start_round_msg(r)
29   assume ¬∃ r', r'', v. r' > r ∧ join_ack_msg(n, r', r'', v)
30   # find maximal round in which n voted, and the corresponding vote.
31   # maxr = ⊥ and v is arbitrary when n never voted.
32   local maxr, v := max {(r', v') | vote_msg(n, r', v') ∧ r' < r}
33   join_ack_msg(n, r, maxr, v) := true
34 }
35 action PROPOSE(r : round, q : quorum) {
36   assume r ≠ ⊥
37   assume ∀ v. ¬propose_msg(r, v)
38   # 1b from quorum q
39   assume ∀ n. member(n, q) → ∃ r', v. join_ack_msg(n, r, r', v)
40   # find the maximal round in which a node in the quorum reported

```

```

41   # voting, and the corresponding vote.
42   # v is arbitrary if the nodes reported not voting.
43   local maxr, v := max {(r', v') | ∃ n. member(n, q)
44                         ∧ join_ack_msg(n, r, r', v') ∧ r' ≠ ⊥}
45   propose_msg(r, v) := true # propose value v
46 }
47 action VOTE(n : node, r : round, v : value) {
48   assume r ≠ ⊥
49   assume propose_msg(r, v)
50   assume ¬∃ r', r'', v. r' > r ∧ join_ack_msg(n, r', r'', v)
51   vote_msg(n, r, v) := true
52 }
53 action LEARN(n : node, r : round, v : value, q : quorum) {
54   assume r ≠ ⊥
55   # 2b from quorum q
56   assume ∀ n. member(n, q) → vote_msg(n, r, v)
57   decision(n, r, v) := true
58 }

```

$\forall n_1, n_2 : \text{node}, r_1, r_2 : \text{round}, v_1, v_2 : \text{value}. \text{decision}(n_1, r_1, v_1) \wedge \text{decision}(n_2, r_2, v_2) \rightarrow v_1 = v_2$
 $\forall r : \text{round}, v_1, v_2 : \text{value}. \text{propose_msg}(r, v_1) \wedge \text{propose_msg}(r, v_2) \rightarrow v_1 = v_2$
 $\forall n : \text{node}, r : \text{round}, v : \text{value}. \text{vote_msg}(n, r, v) \rightarrow \text{propose_msg}(r, v)$
 $\forall r : \text{round}, v : \text{value}. (\exists n : \text{node}. \text{decision}(n, r, v)) \rightarrow \exists q : \text{quorum}. \forall n : \text{node}. \text{member}(n, q) \rightarrow \text{vote_msg}(n, r, v)$
 $\forall n : \text{node}, r, r' : \text{round}, v, v' : \text{value}. \text{join_ack_msg}(n, r, \perp, v) \wedge r' < r \rightarrow \neg \text{vote_msg}(n, r', v')$
 $\forall n : \text{node}, r, r' : \text{round}, v : \text{value}. \text{join_ack_msg}(n, r, r', v) \wedge r' \neq \perp \rightarrow r' < r \wedge \text{vote_msg}(n, r', v)$
 $\forall n : \text{node}, r, r', r'' : \text{round}, v, v' : \text{value}. \text{join_ack_msg}(n, r, r', v) \wedge r' \neq \perp \wedge r' < r'' < r \rightarrow \neg \text{vote_msg}(n, r'', v')$
 $\forall n : \text{node}, v : \text{value}. \neg \text{vote_msg}(n, \perp, v)$
 $\forall r_1, r_2 : \text{round}, v_1, v_2 : \text{value}, q : \text{quorum}. \text{propose_msg}(r_2, v_2) \wedge r_1 < r_2 \wedge v_1 \neq v_2 \rightarrow$
 $\exists n : \text{node}, r', r'' : \text{round}, v : \text{value}. \text{member}(n, q) \wedge \neg \text{vote_msg}(n, r_1, v_1) \wedge r' > r_1 \wedge \text{join_ack_msg}(n, r', r'', v)$

VCs in first-order logic

Challenge: encoding into EPR

- But what can you possibly express in such a restricted logic?
 - Transitive closure over deterministic paths
 - Set cardinalities
 - Avoiding quantifier alternations
 - Encoding liveness and LTL

Quantifier alternation cycles

- Axiom

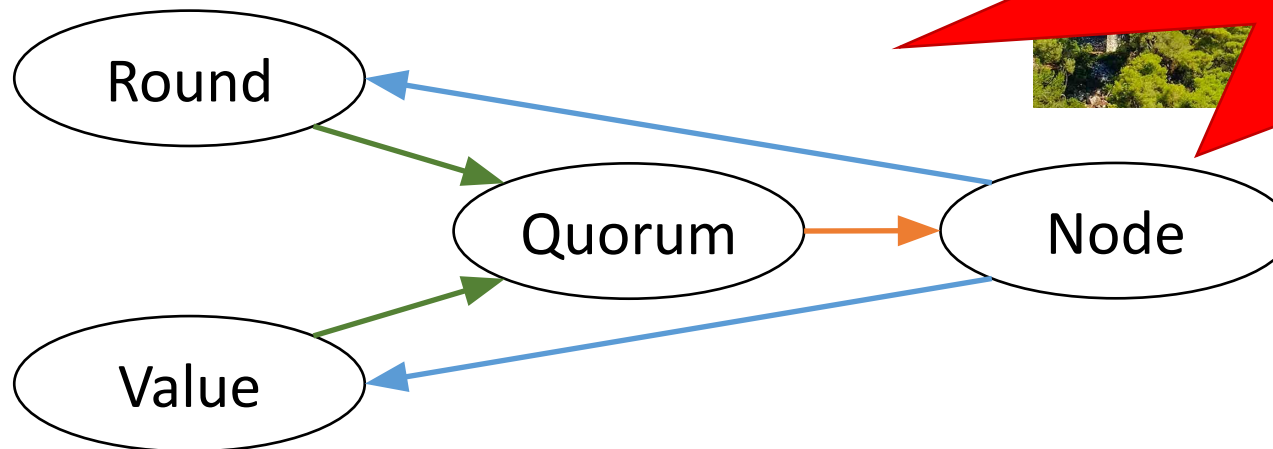
$\forall q_1, q_2: \text{Quorum}. \exists n: \text{Node}. \dots$

- Action precondition

$\dots \forall n: \text{Node}. \dots \rightarrow \exists r: \text{Round}, v: \text{Value}. \dots$

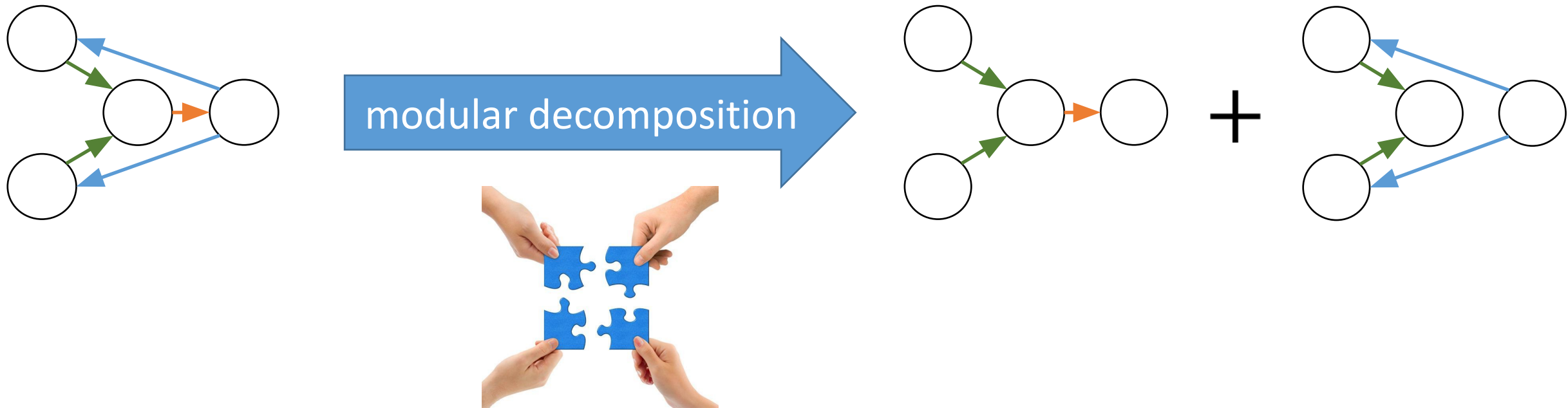
- Inductive invariant

$\forall r: \text{Round}, v: \text{Value}. \dots \rightarrow \exists q: \text{Quorum}. \dots$



Quantifier Alternation Cycle

Modularity



Inductive invariant of Paxos

safety property

invariant decision(N1,R1,V1) & decision(N2,R2,V2) -> V1 = V2

proposals are unique per round

invariant proposal(R,V1) & proposal(R,V2) -> V1 = V2

only vote for proposed values

invariant vote(N,R,V) -> proposal(R,V)

decisions come from quorums of votes:

invariant forall R, V. (exists N. decision(N,R,V)) -> exists Q. forall N. member(N, Q) -> vote(N,R,V)

properties of one_b_max_vote

invariant one_b_max_vote(N,R2,none,V1) & ~le(R2,R1) -> ~vote(N,R1,V2)

invariant one_b_max_vote(N,R,RM,V) & RM ~= none -> ~le(R,RM) & vote(N,RM,V)

invariant one_b_max_vote(N,R,RM,V) & RM ~= none & ~le(R,R0) & ~le(R0,RM) -> ~vote(N,R0,V0)

property of choosable and proposal

invariant ~le(R2,R1) & proposal(R2,V2) & V1 ~= V2 -> exists N. member(N,Q) & left_rnd(N,R1) & ~vote(N,R1,V1)

property of one_b, left_rnd

invariant one_b(N,R2) & ~le(R2,R1) -> left_rnd(N,R1)

Paxos made EPR: experimental evaluation

Protocol	Model [LOC]	Invariant [Conjectures]		
Paxos	85	11	2.2	0.1
Multi-Paxos	98	12	2.6	0.1
Vertical Paxos*	123	18	2.2	0.2
Fast Paxos*	117	17	6.2	1.6
Flexible Paxos	88	11	2.2	0
Stoppable Paxos*	132	16	5.4	0.9

*first mechanized verification

Transformation to EPR reusable across all variants!

Paxos made EPR: experimental evaluation

Protocol	Model [LOC]	Invariant [Conjectures]		
Paxos	85	11	2.2	0.1
Multi-Paxos	98	12	2.6	0.1
Vertical Paxos*	123	18	2.2	0.2
Fast Paxos*	117	17	6.2	1.6
Flexible Paxos	88	11	2.2	0
Stoppable Paxos*	132	16	5.4	0.9

Proof / code ratio:

IronFleet: ~4

Verdi: ~10

Ivy: ~0.2

*first mechanized verification

Transformation to EPR reusable across all variants!

Paxos made EPR: experimental evaluation

Protocol	Model [LOC]	Invariant [Conjectures]		
Paxos	85	11	2.2	0.1
Multi-Paxos	98	12	2.6	0.1
Vertical Paxos*	123	18	2.2	0.2
Fast Paxos*	117	17	6.2	1.6
Flexible Paxos	88	11	2.2	0
Stoppable Paxos*	132	16	5.4	0.9

μ – mean

σ – std. deviation

*first mechanized verification

Transformation to EPR reusable across all variants!

Paxos made EPR: experimental evaluation

Protocol	Model [LOC]	Invariant [Conjectures]		
Paxos	85	11	2.2	0.1
Multi-Paxos	98	12	2.6	0.1
Vertical Paxos*	123	18	2.2	0.2
Fast Paxos*	117	17	6.2	1.6
Flexible Paxos	88	11	2.2	0
Stoppable Paxos*	132	16	5.4	0.9

Rounds			T.O.
2	1.2	0.1	0
4	1.8	0.4	0
8	107	129	30%
16	229	110	70%

Multi-Paxos in FOL

*first mechanized verification

Transformation to EPR reusable across all variants!

Paxos made EPR: experimental evaluation

Protocol	Model [LOC]	Invariant [Conjectures]		
Paxos	85	11	2.2	0.1
Multi-Paxos	98	12	2.6	0.1
Vertical Paxos*	123	18	2.2	0.2
Fast Paxos*	117	17	6.2	1.6
Flexible Paxos	88	11	2.2	0
Stoppable Paxos*	132	16	5.4	0.9

Rounds			T.O.
2	186	123	50%
4	300	0	100%
8	300	0	100%
16	300	0	100%

Stoppable Paxos in FOL

*first mechanized verification

Transformation to EPR reusable across all variants!

Challenge: encoding into EPR

- But what can you possibly express in such a restricted logic?
 - Transitive closure over deterministic paths
 - Set cardinalities
 - Avoiding quantifier alternations
 - Encoding liveness and LTL

Liveness properties

- Liveness property: “something good eventually happens”
- Often depend on fairness assumptions
- Typically proven by ranking functions, well-founded relations
 - Well beyond EPR, or not?

[POPL'18] O. P., Jochen Hoenicke, Giuliano Losa, Andreas Podelski, Mooly Sagiv, Sharon Shoham.

Reducing Liveness to Safety in First-Order Logic

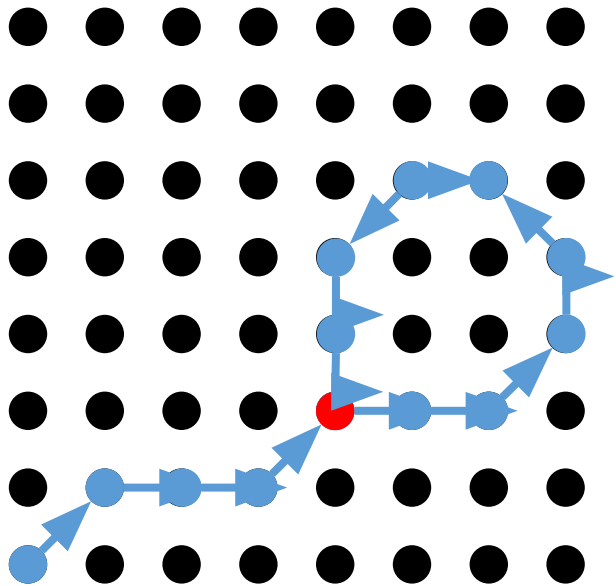
[FMCAD'18] O. P., Jochen Hoenicke, Kenneth L. McMillan, Andreas Podelski, Mooly Sagiv, Sharon Shoham.

Temporal Prophecy for Proving Temporal Properties of Infinite-State Systems.

Lasso & Dynamic Abstraction

Finite State

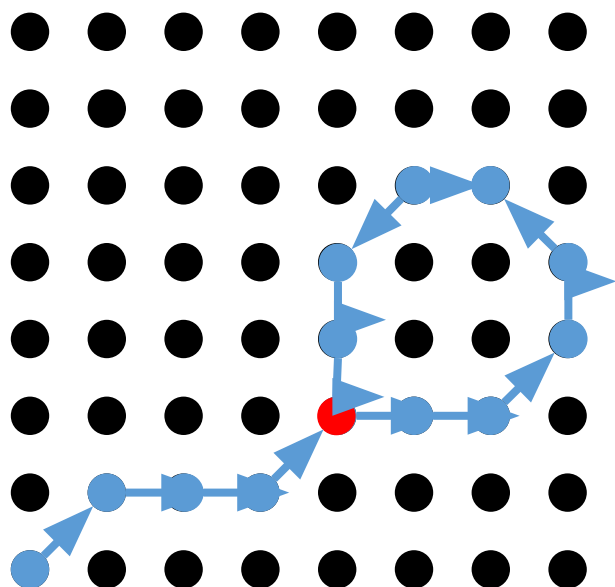
Parameterized



Liveness $\Leftrightarrow$ No Lasso

Lasso & Dynamic Abstraction

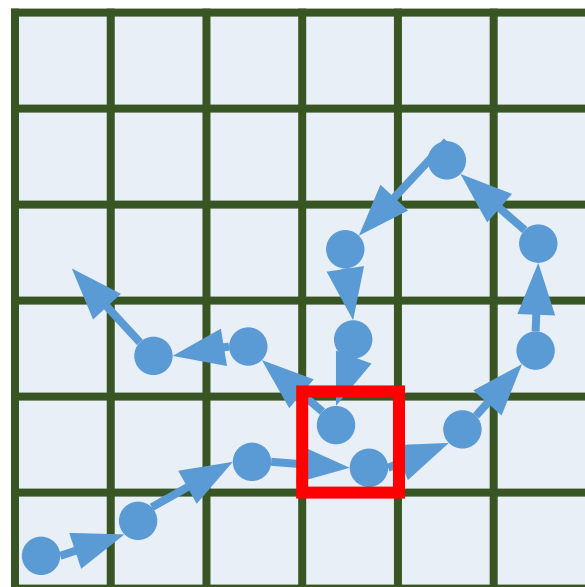
Finite State Parameterized



Liveness $\Leftrightarrow$ No Lasso

Infinite State

Finite Abstraction

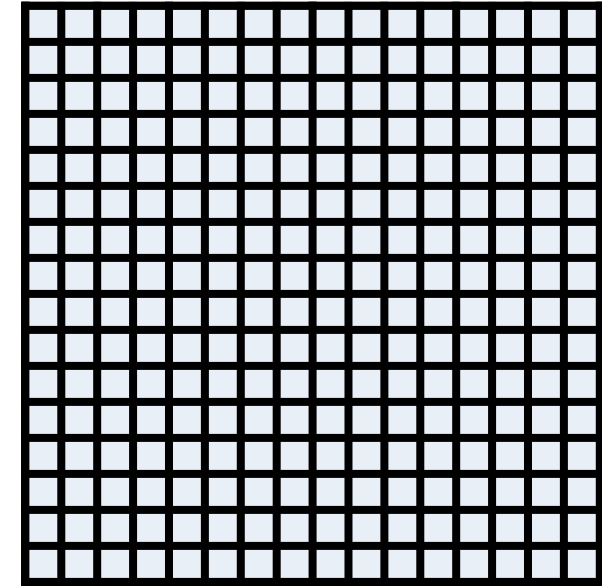


Liveness $\Leftarrow$ No Lasso
Problem: Spurious Lasso

All expressible in EPR

[illegible][illegible]

Dynamic Abstraction

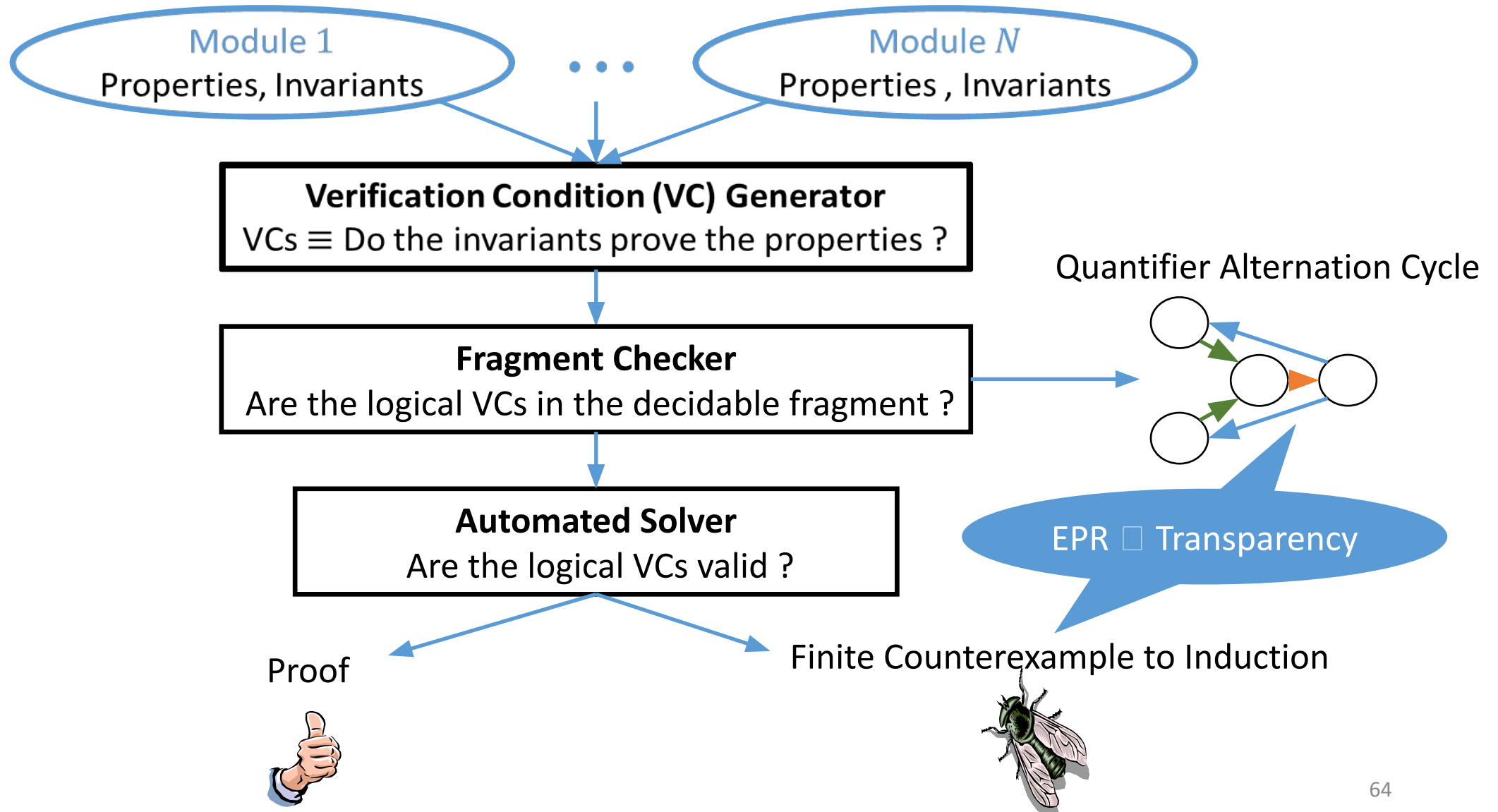


Liveness $\Leftarrow$ No Lasso
Fewer Spurious Lassos

Challenge: encoding into EPR

- But what can you possibly express in such a restricted logic?
 - Transitive closure over deterministic paths
 - Set cardinalities
 - Avoiding quantifier alternations
 - Encoding liveness and LTL

Logic-based deductive verification in Ivy



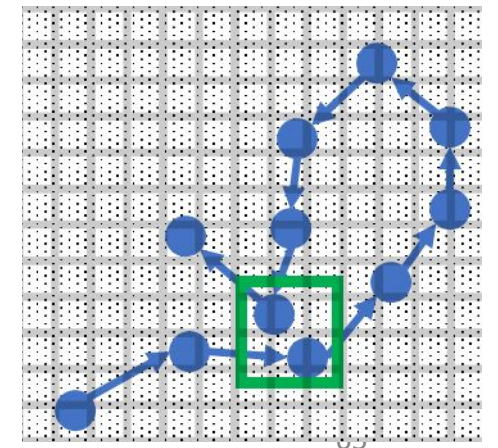
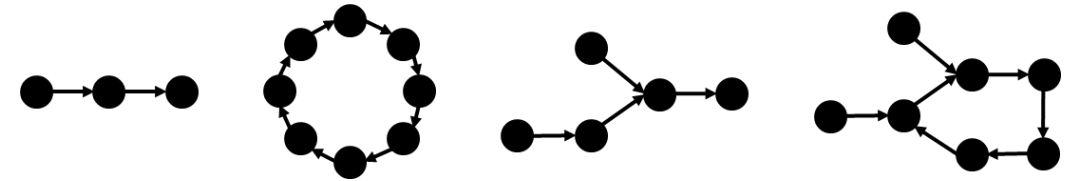
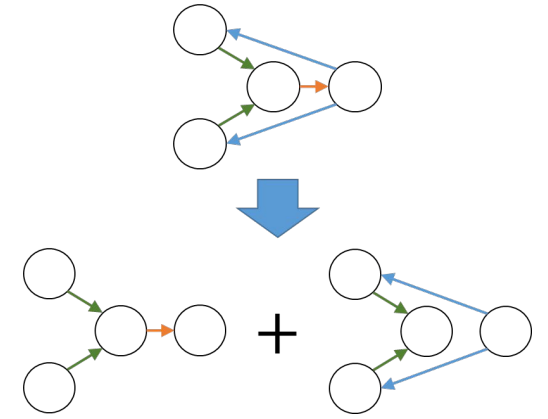
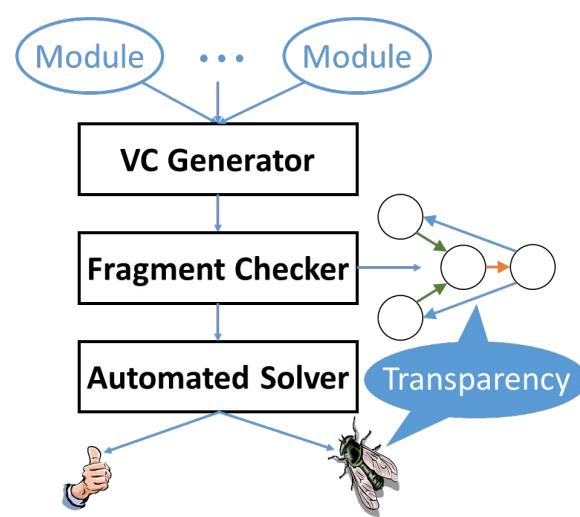
Part 1: Conclusion

- Verification in EPR

- Deduction is decidable
- Finite counterexamples
- Transparent failures

- Powerful encodings

- Transitive closure for deterministic paths
- Set cardinalities
- Modular decomposition to battle $\forall\exists$ cycles
- Liveness and temporal properties



Part 1: Open Questions

- What cannot be proven with EPR?
 - Proof theory
- Why are solvers stable on EPR?
 - Can it be generalized?
 - Part of why DPLL/CDCL works in practice
- Can it be used beyond a small group of fans?
 - Transitive closure 😊, LTL 😊, Cardinalities 😊, Breaking $\forall\exists$ cycles 😞
- Can you get the benefit of EPR without the costs?
 - Bounded quantifier instantiations
 - Special case of a more general “simple proofs” principle?

Verification of Distributed Protocols: Decidable Modeling and Invariant Inference

Part 1: Decidable Modeling

- The Ivy deductive verification system
- The many-sorted EPR fragment
- Main challenge: expressing interesting systems and properties in EPR
 - Expressing transitive closure
 - Expressing sets and cardinalities
 - Liveness and temporal verification

Part 2: Invariant Inference

- Problem setting

Problem setting

Several Ivy papers provide:

- A collection of examples with manually written invariants
- Invariant checking is decidable, typically ~1s, complex protocols ~10s
- Counterexamples are finite
- Invariants aren't too large or too complex



Protocol	Model [LOC]	Invariant [Conjectures]	EPR [sec]	
			μ	σ
Paxos	85	11	2.2	0.1
Multi-Paxos	98	12	2.6	0.1
Vertical Paxos	123	18	2.2	0.2
Fast Paxos	117	17	6.2	1.6
Flexible Paxos	88	11	2.2	0
Stoppable Paxos	132	16	5.4	0.9

```
# safety property
invariant decision(N1,R1,V1) & decision(N2,R2,V2) -> V1 = V2

# proposals are unique per round
invariant proposal(R,V1) & proposal(R,V2) -> V1 = V2

# only vote for proposed values
invariant vote(N,R,V) -> proposal(R,V)

# decisions come from quorums of votes:
invariant forall R, V. (exists N. decision(N,R,V)) -> exists Q. forall N. member(N, Q) -> vote(N,R,V)

# properties of one_b_max_vote
invariant one_b_max_vote(N,R2,none,V1) & ~le(R2,R1) -> ~vote(N,R1,V2)
invariant one_b_max_vote(N,R,RM,V) & RM ~= none -> ~le(R,RM) & vote(N,RM,V)
invariant one_b_max_vote(N,R,RM,V) & RM ~= none & ~le(R,R0) & ~le(R0,RM) -> ~vote(N,R0,V0)

# property of choosable and proposal
invariant ~le(R2,R1) & proposal(R2,V2) & V1 ~= V2 -> exists N. member(N,Q) & left_rnd(N,R1) & ~vote(N,R1,V1)

# property of one_b, left_rnd
invariant one_b(N,R2) & ~le(R2,R1) -> left_rnd(N,R1)
```

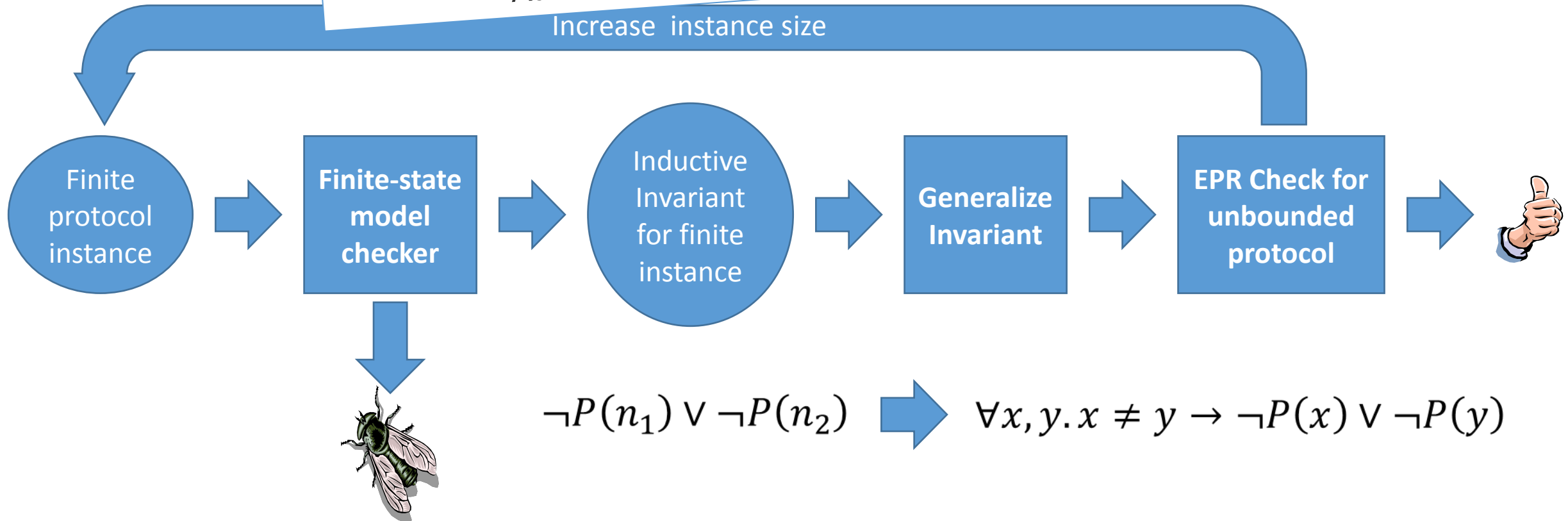
- UPDR** [CAV'15, JACM'17] Aleksandr Karbyshev, Nikolaj Bjorner, Shachar Itzhaky, Noam Rinetzkky and Sharon Shoham.
Property-Directed Inference of Universal Invariants or Proving Their Absence.
- I4** [SOSP'19] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Karem Sakallah.
I4: incremental inference of inductive invariants for verification of distributed protocols.
- FOL-IC3** [PLDI'20] Jason Koenig, O. P., Neil Immerman, and Alex Aiken.
First-Order Quantified Separators.
- SWISS** [NSDI'21] Travis Hance, Marijn Heule, Ruben Martins, and Bryan Parno.
Finding Invariants of Distributed Systems: It's a Small (Enough) World After All.
- DistAI** [OSDI'21] Jianan Yao, Runzhou Tao, Ronghui Gu, Jason Nieh, Suman Jana, and Gabriel Ryan.
DistAI: Data-Driven Automated Invariant Learning for Distributed Protocols.
- IC3PO** [NSF'21] Aman Goel and Karem Sakallah.
On Symmetry and Quantification: A New Approach to Verify Distributed Protocols.
- FOL-IC3** [TACAS'22] Jason Koenig, O. P., Sharon Shoham, and Alex Aiken.
Inferring Invariants with Quantifier Alternations: Taming the Search Space Explosion.
- DuoAI** [OSDI'22] Jianan Yao, Runzhou Tao, Ronghui Gu, and Jason Nieh.
DuoAI: Fast, Automated Inference of Inductive Invariants for Verifying Distributed Protocols.

Interesting ideas

- Generalize from finite instances
 - I4
 - IC3PO
- Explicitly enumerate candidate invariants
 - SWISS
 - DistAI
 - DuoAI
- Generalize IC3/PDR by adapting lemma generation
 - UPDR
 - FOL-IC3

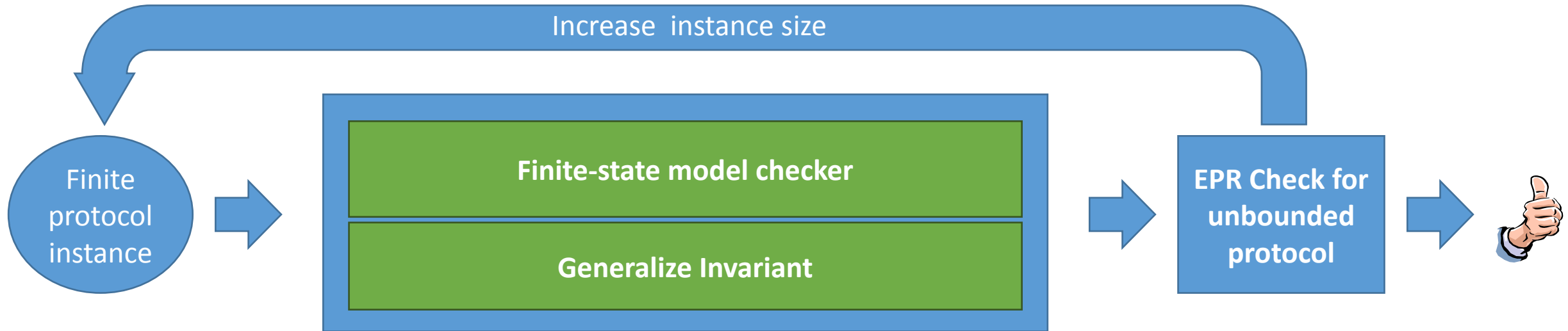
Generalize from finite instances (I4)

[TACAS'01] Amir Pnueli, Sitvanit Ruah, and Lenore Zuck.
Automatic deductive verification with invisible invariants.



[SOSP'19] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Karem Sakallah.
I4: incremental inference of inductive invariants for verification of distributed protocols.

Generalize from finite instances (IC3PO)



$$\neg P(n_1) \vee \neg P(n_2) \Rightarrow \forall x, y. x \neq y \rightarrow \neg P(x) \vee \neg P(y) \Rightarrow \begin{aligned} &(\neg P(n_1) \vee \neg P(n_2)) \wedge \\ &(\neg P(n_1) \vee \neg P(n_3)) \wedge \\ &(\neg P(n_2) \vee \neg P(n_3)) \end{aligned}$$

[NSF'21] Aman Goel and Karem Sakallah.

On Symmetry and Quantification: A New Approach to Verify Distributed Protocols.

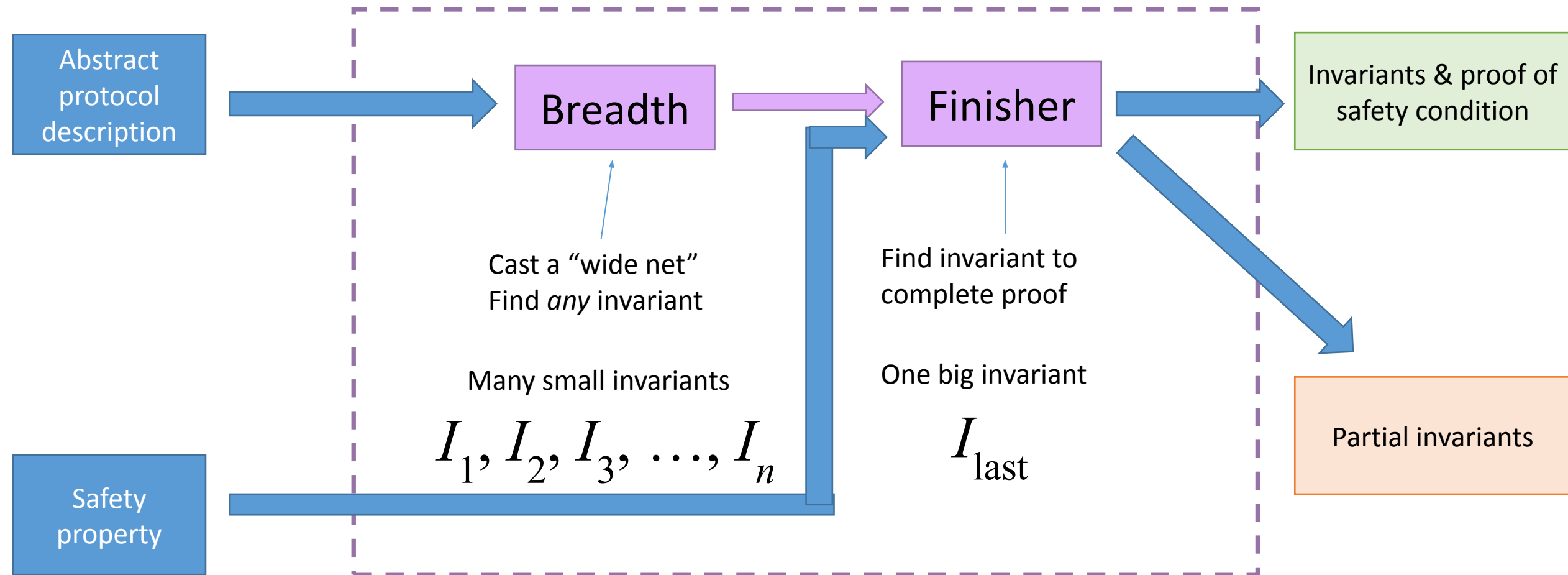
Explicitly enumerate invariants (SWISS)

- Typically $Inv = p_1 \wedge \dots \wedge p_n$
- Each p_i isn't too complex, maybe we can explicitly enumerate all of them?
Would that be useful?
- If we find some p that is inductive, we can learn it as an invariant
 - May help make additional p 's inductive
- Ultimately, we want to prove the safety property

[NSDI'21] Travis Hance, Marijn Heule, Ruben Martins, and Bryan Parno.

Finding Invariants of Distributed Systems: It's a Small (Enough) World After All.

Explicitly enumerate invariants (SWISS)



[NSDI'21] Travis Hance, Marijn Heule, Ruben Martins, and Bryan Parno.

Finding Invariants of Distributed Systems: It's a Small (Enough) World After All.

Explicitly enumerate invariants (SWISS)

Exploring the space of candidate invariant predicates for Paxos

	Candidate invariant space	Number of candidate invariants	Symmetries	Counter-example filters	Removing redundant invariants	Invariant predicates
Finisher	6 terms	~ 99,000,000,000,000	~ 200,000,000,000	155	155	5
Breadth	3 terms	~ 820,000,000	~ 3,000,000	~ 900,000	2,250	801

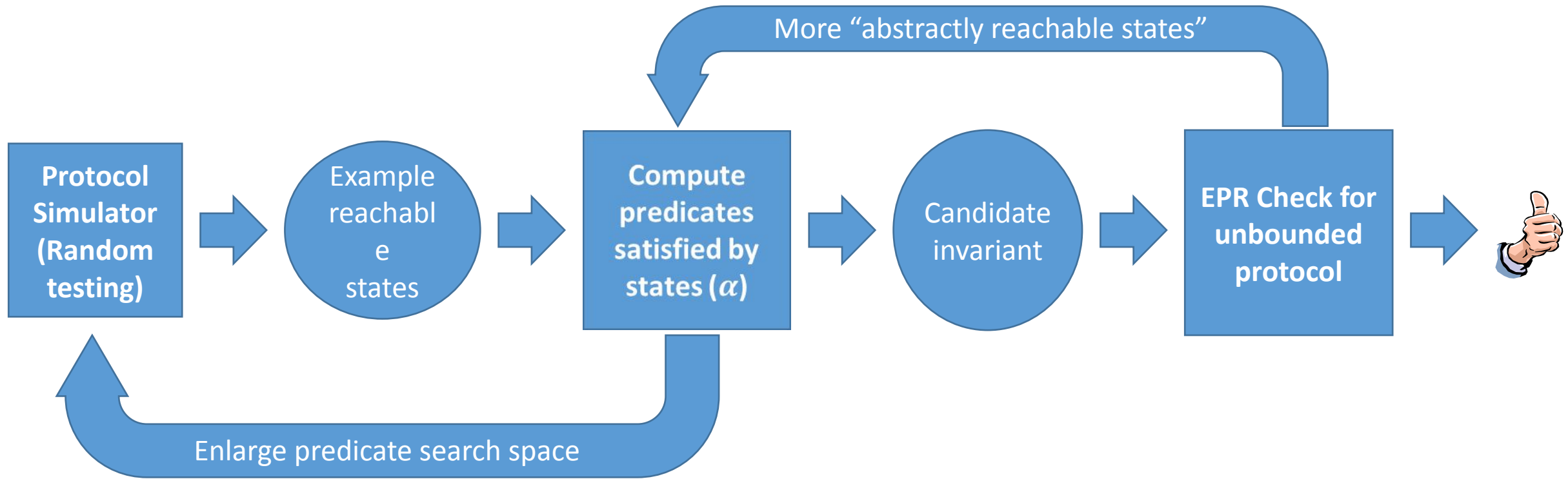
100 ms on average
Brute force is not feasible

Counterexample-guided synthesis:
When one predicate fails to be inductive, use it to narrow your search space.

[NSDI'21] Travis Hance, Marijn Heule, Ruben Martins, and Bryan Parno.

Finding Invariants of Distributed Systems: It's a Small (Enough) World After All.

Explicitly enumerate invariants (DistAI, DuoAI)



[OSDI'21] Jianan Yao, Runzhou Tao, Ronghui Gu, Jason Nieh, Suman Jana, and Gabriel Ryan.
DistAI: Data-Driven Automated Invariant Learning for Distributed Protocols.

[OSDI'22] Jianan Yao, Runzhou Tao, Ronghui Gu, and Jason Nieh.
DuoAI: Fast, Automated Inference of Inductive Invariants for Verifying Distributed Protocols.

Adapt IC3/PDR (UPDR, FOL-IC3)

- Core task for lemma learning in IC3/PDR:
 - Given $F = p_1 \wedge \dots \wedge p_n$ and state s , find p s.t. $s \not\models p$ and $\text{post}([F] \cap [p]) \subseteq [p]$
- UPDR: for p a forall-only formula, the diagram of s can be used
- FOL-IC3:
 - Closely related problem is **separation**: given $s_1^+, \dots, s_n^+$ and $s_1^-, \dots, s_m^-$ find p such that $s_1^+, \dots, s_n^+ \models p$ and $s_1^-, \dots, s_m^- \models \neg p$
 - For forall-exists formulas, separation is NP-complete
 - Use a SAT solver!
 - Many more techniques to scale to complex protocols

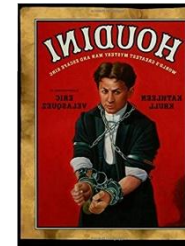
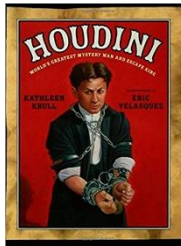
[CAV'15, JACM'17] Aleksandr Karbyshev, Nikolaj Bjorner, Shachar Itzhaky, Noam Rinetzky and Sharon Shoham.
Property-Directed Inference of Universal Invariants or Proving Their Absence.

[PLDI'20] Jason Koenig, O. P., Neil Immerman, and Alex Aiken.
First-Order Quantified Separators.

[TACAS'22] Jason Koenig, O. P., Sharon Shoham, and Alex Aiken.
Inferring Invariants with Quantifier Alternations: Taming the Search Space Explosion.

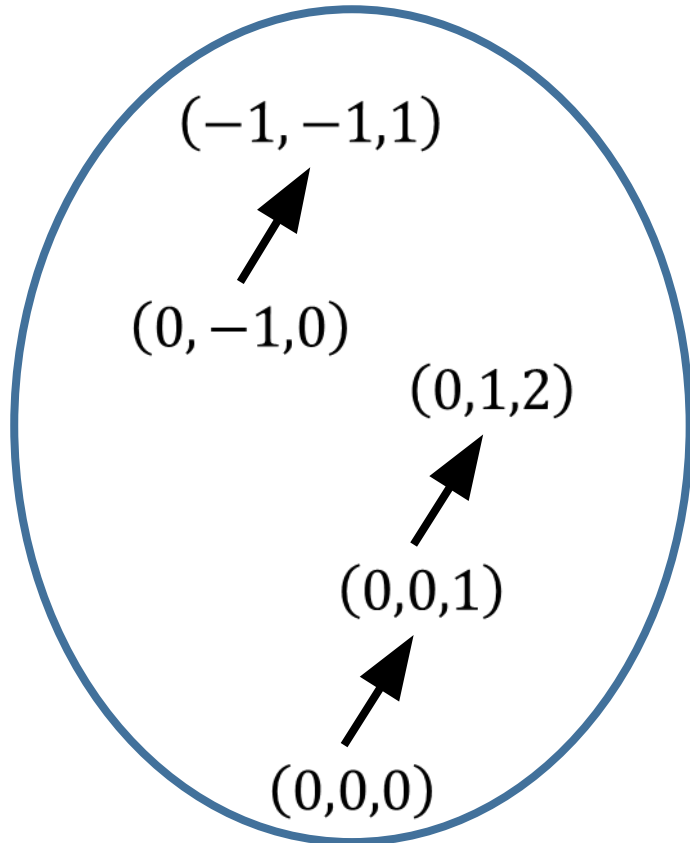
Induction Duality and Primal-Dual Houdini

- Induction Duality
 - Formal symmetric connection between execution traces and incremental induction proofs
- Primal-Dual Houdini



- Houdini + Dual Houdini = Primal-dual invariant inference
- Interesting theoretical properties
- Promising empirical results for distributed protocols

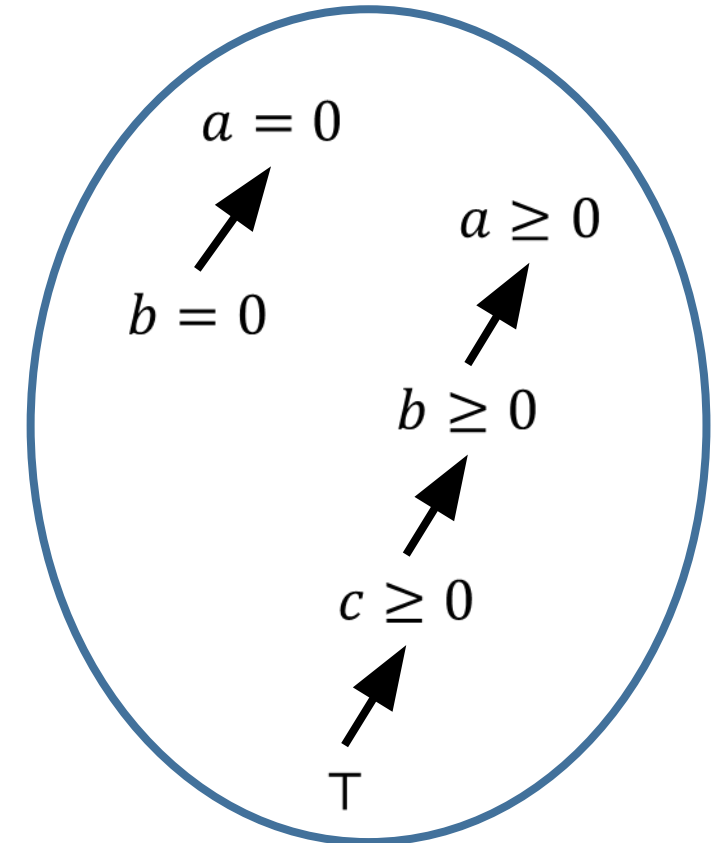
Execution vs Incremental Induction



States and
Transitions

```
a, b, c := 0, 0, 0
while * {
  assert a ≥ 0
  a, b, c := a+b, b+c, c+1
}
```

Loop invariant:
 $a \geq 0 \wedge b \geq 0 \wedge c \geq 0$



Predicates and
Incremental Induction

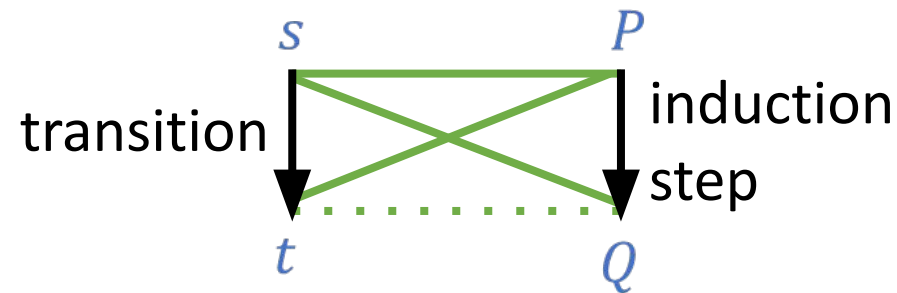
Formalizing symmetric incremental induction

- Intuition: if P is invariant, Q is also invariant

- Formally:

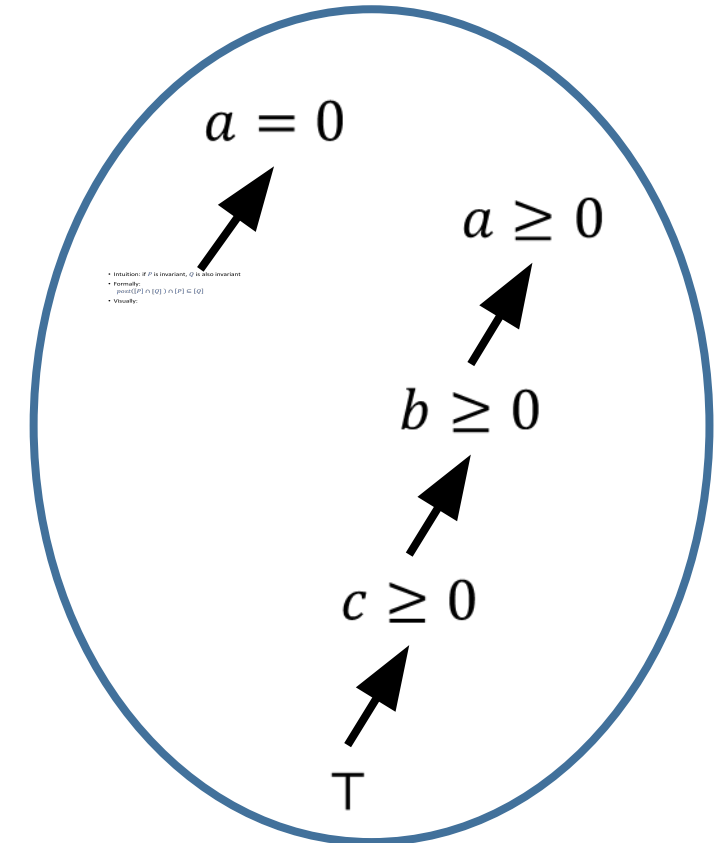
$$post([P] \cap [Q]) \cap [P] \subseteq [Q] \Leftrightarrow P \wedge Q \models wp(P \rightarrow Q)$$

- Visually:



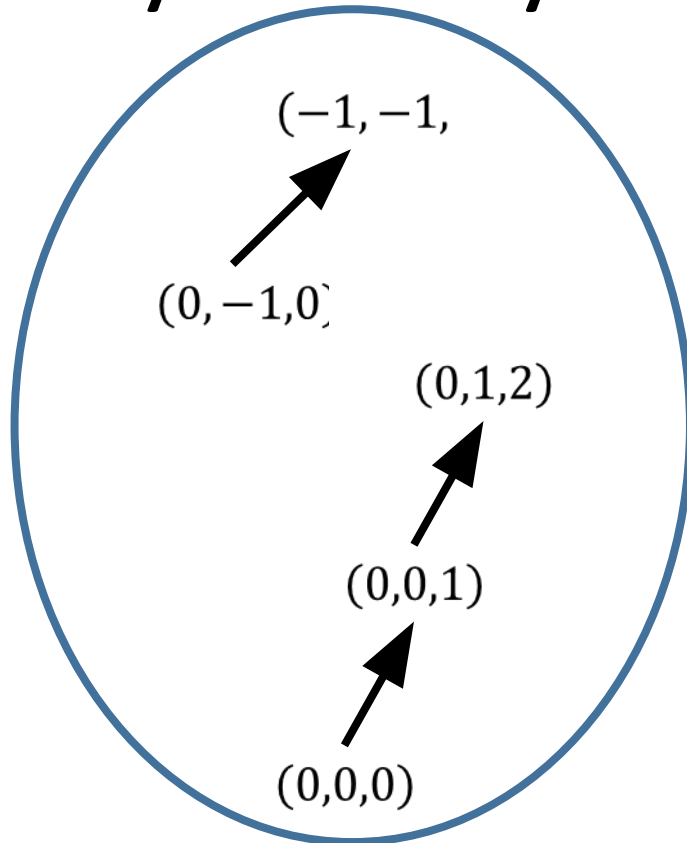
- Contrast with IC3/PDR relative inductiveness:

$$post([P] \cap [Q]) \subseteq [Q]$$

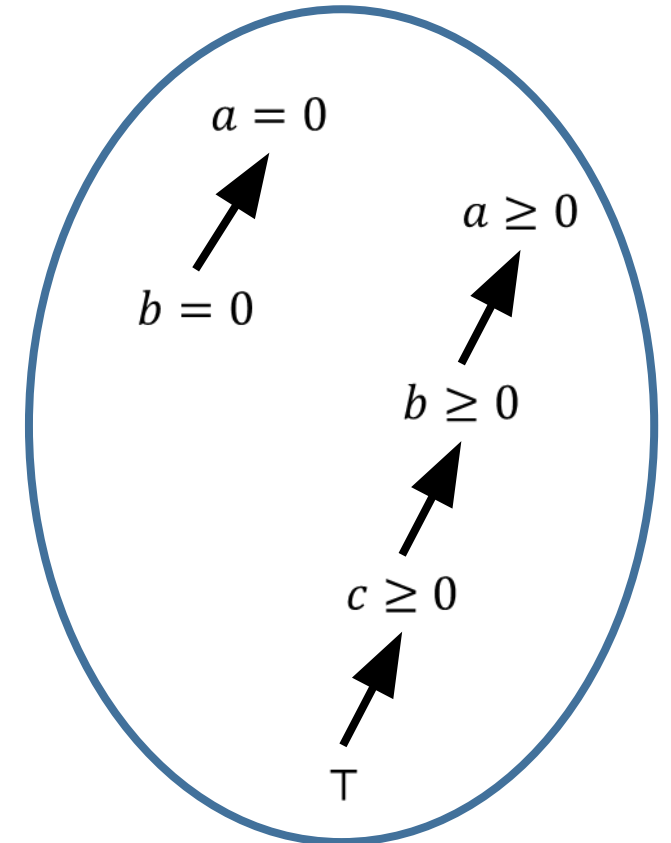
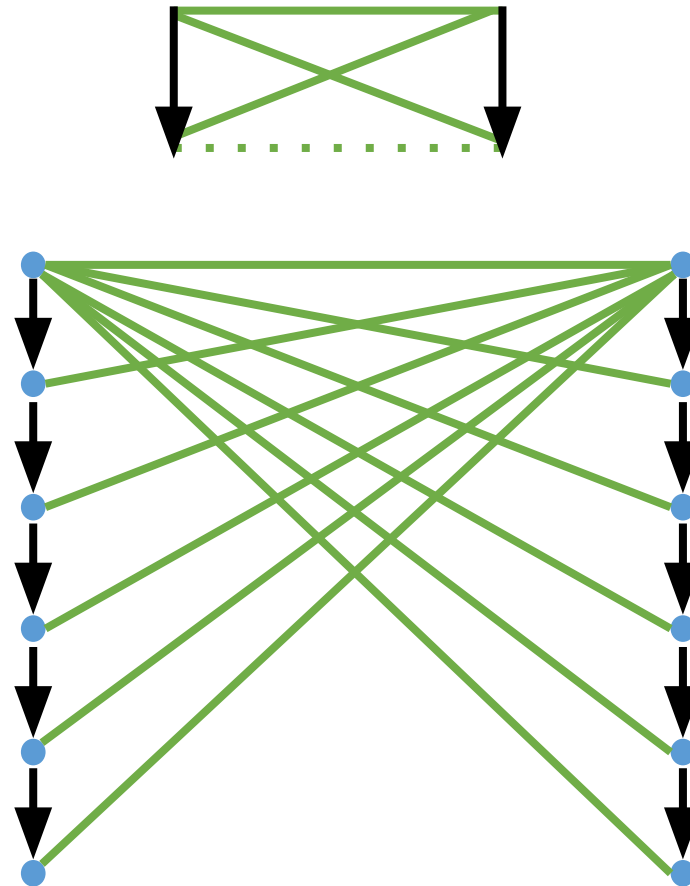


Predicates and
Incremental Induction

Executions vs Incremental Induction: Symmetry

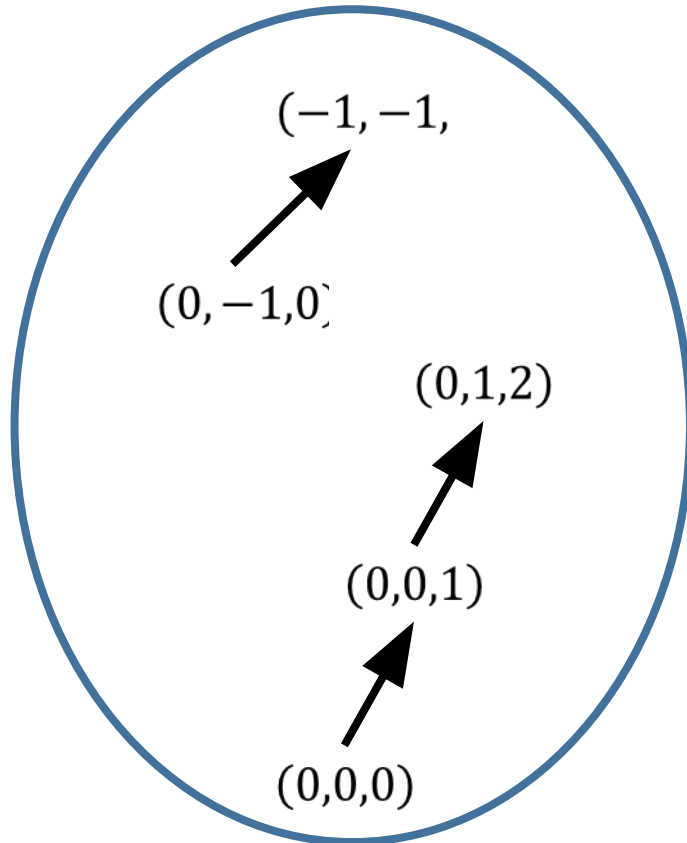


States and Transitions

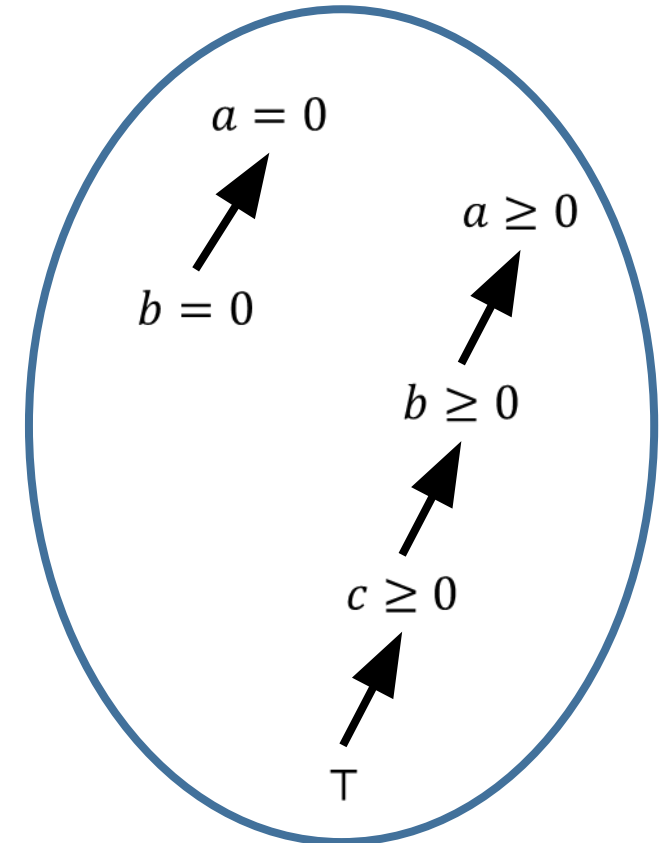
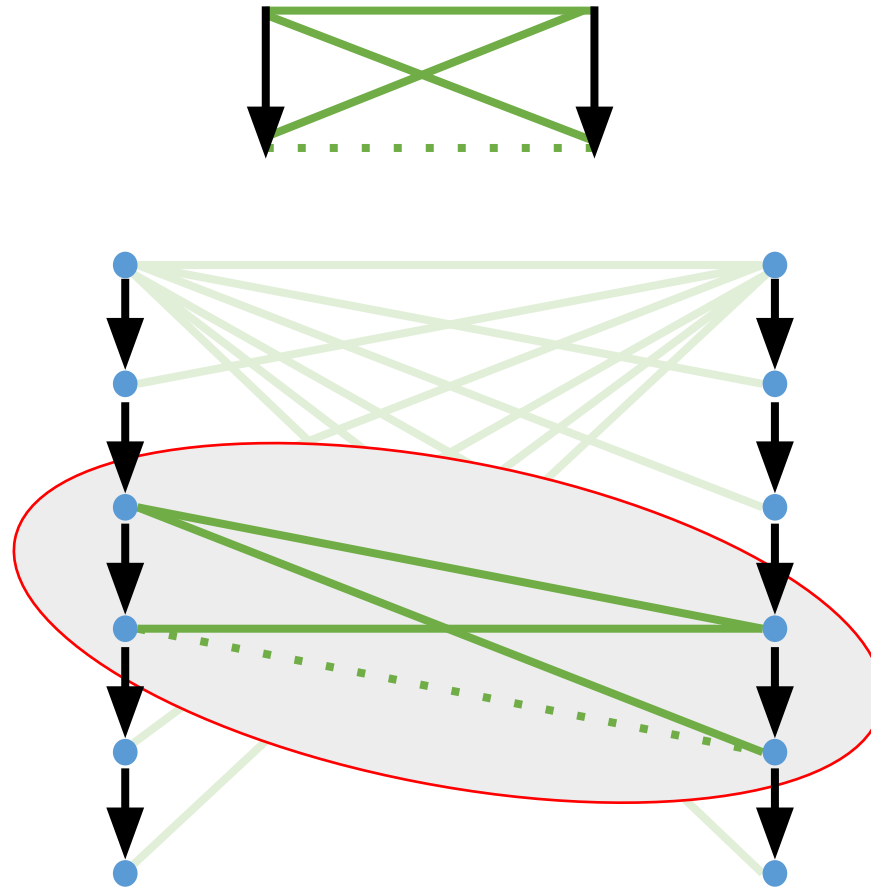


Predicates and
Incremental Induction Steps

Executions vs Incremental Induction: Symmetry

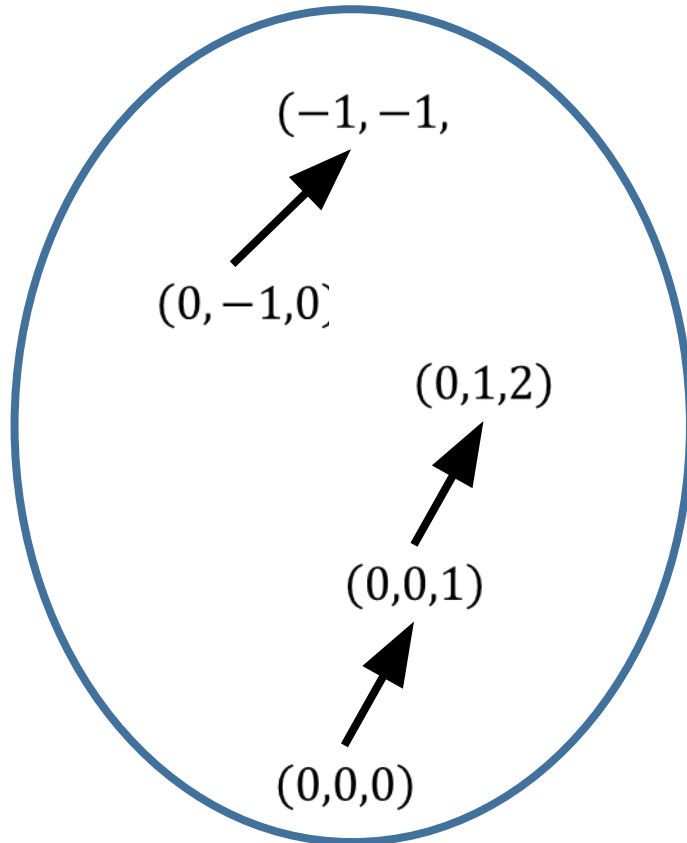


States and Transitions

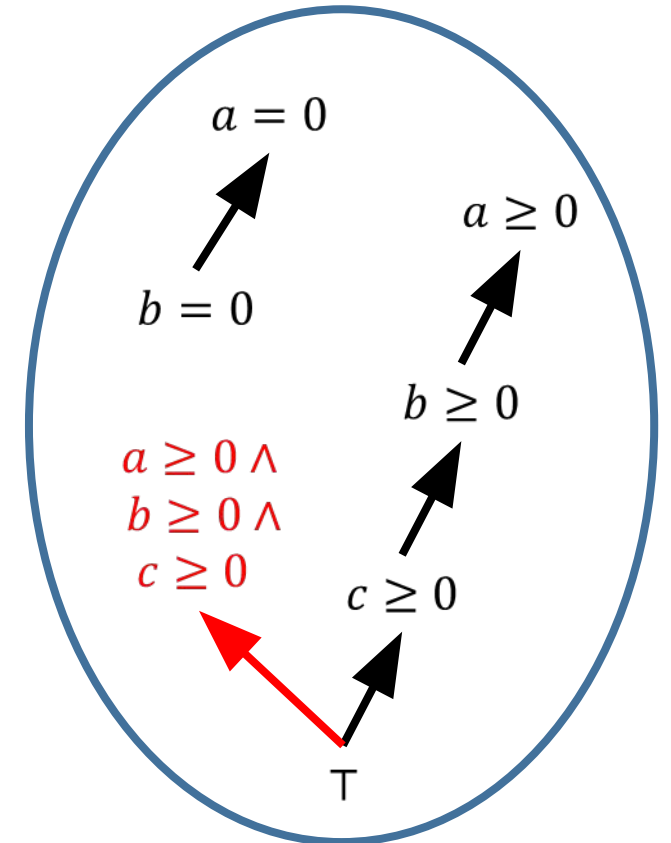
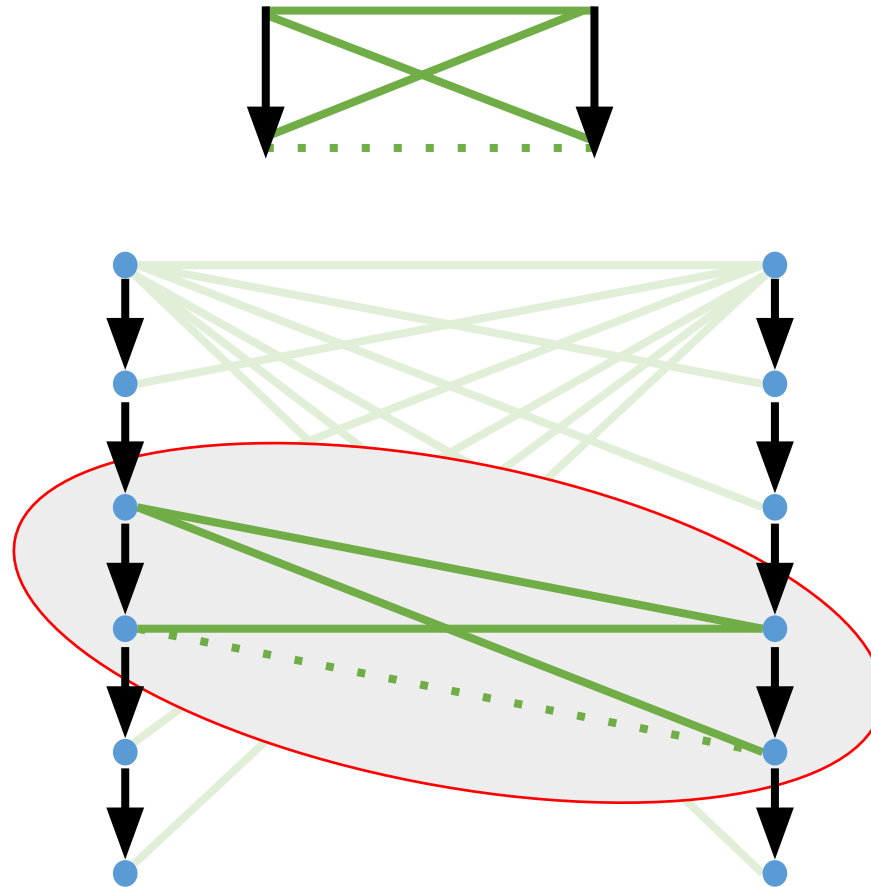


Predicates and
Incremental Induction Steps

Executions vs Incremental Induction: Symmetry



States and Transitions



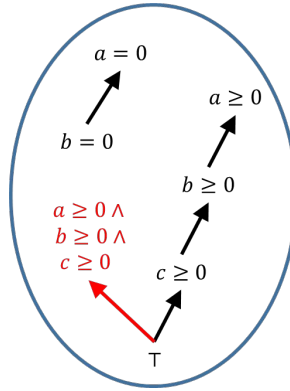
Predicates and
Incremental Induction Steps

Induction Duality Structure

V : finite sets of states



V' : finite sets of predicates



Induction over bounded proofs

$$E_\omega = \{(S, S \cup T) \in V \times V \mid \dots\}$$

Dual-Inductive

$\supseteq$

Induction over executions

$$E'_\omega = \{(P, P \cup Q) \in V' \times V' \mid \dots\}$$

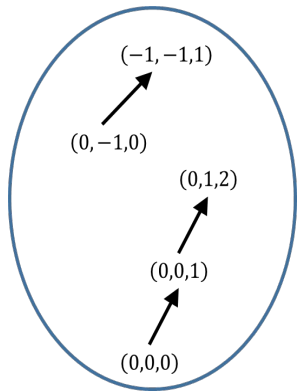
Inductive

$\supseteq$

Transitions

$$E = \{(S, S \cup \{t\}) \in V \times V \mid t \in \text{post}(S)\}$$

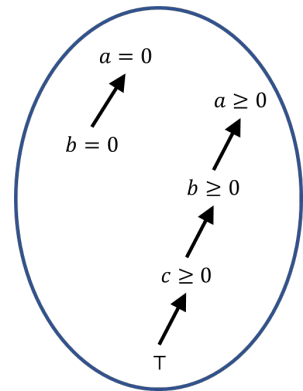
Reachable



Bounded incremental induction

$$E' = \{(P, P \cup Q) \in E'_\omega \mid |Q| \leq k\}$$

k-provable





Induction over bounded proofs

$$E_{\omega} = \{(S, S \cup T) \in V \times V \mid \dots\}$$

Dual-Inductive

$\supseteq$

Transitions

$$E = \{(S, S \cup \{t\}) \in V \times V \mid t \in \text{post}(S)\}$$

Reachable

Induction over executions

$$E'_{\omega} = \{(P, P \cup Q) \in V' \times V' \mid \dots\}$$

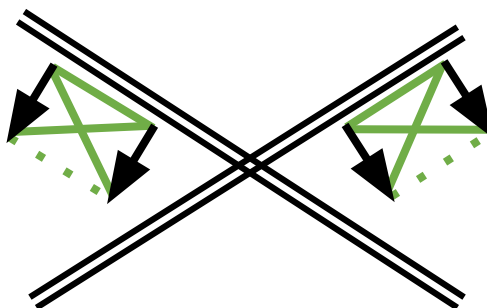
Inductive

$\supseteq$

Bounded incremental induction

$$E' = \{(P, P \cup Q) \in E'_{\omega} \mid |Q| \leq k\}$$

k-provable



Induction over bounded proofs

$$E_{\omega} = \{(S, S \cup T) \in V \times V \mid \dots\}$$

Dual-Inductive

UI

Transitions

$$E = \{(S, S \cup \{t\}) \in V \times V \mid t \in \text{post}(S)\}$$

Reachable

Induction over executions

$$E'_{\omega} = \{(P, P \cup Q) \in V' \times V' \mid \dots\}$$

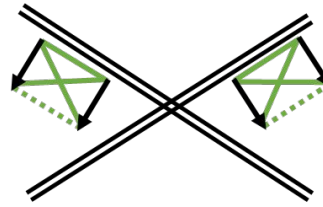
Inductive

UI

Bounded incremental induction

$$E' = \{(P, P \cup Q) \in E'_{\omega} \mid |Q| \leq k\}$$

k-provable



Concept	Induction dual	$\xleftrightarrow[\alpha]{\gamma}$ dual
	k-provable	invariant
	reachable	k-abstractly-reachable
	k-abstractly-reachable	reachable
	Invariant	k-provable
	dual-inductive	
	inductive	

Induction over bounded proofs
 $E_\omega = \{(S, S \cup T) \in V \times V \mid \dots\}$

Dual-Inductive

UI

Transitions

$E = \{(S, S \cup \{t\}) \in V \times V \mid t \in \text{post}(S)\}$

Reachable

Induction over executions

$E'_\omega = \{(P, P \cup Q) \in V' \times V' \mid \dots\}$

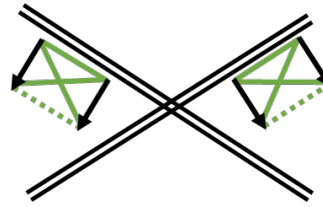
Inductive

UI

Bounded incremental induction

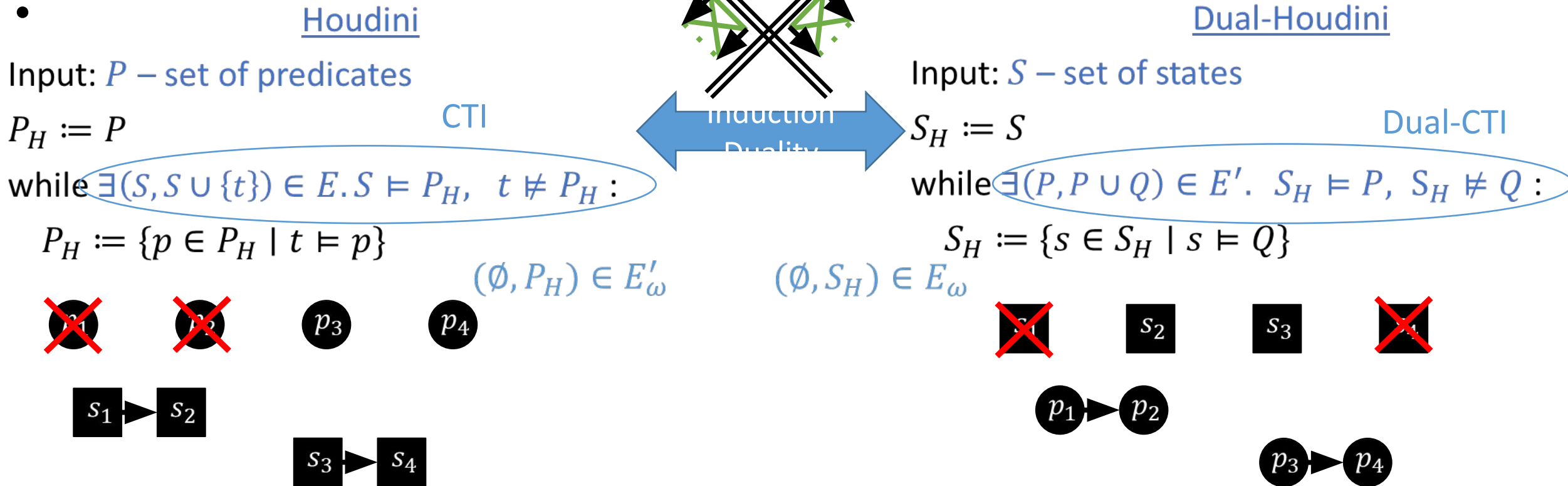
$E' = \{(P, P \cup Q) \in E'_\omega \mid |Q| \leq k\}$

k-provable



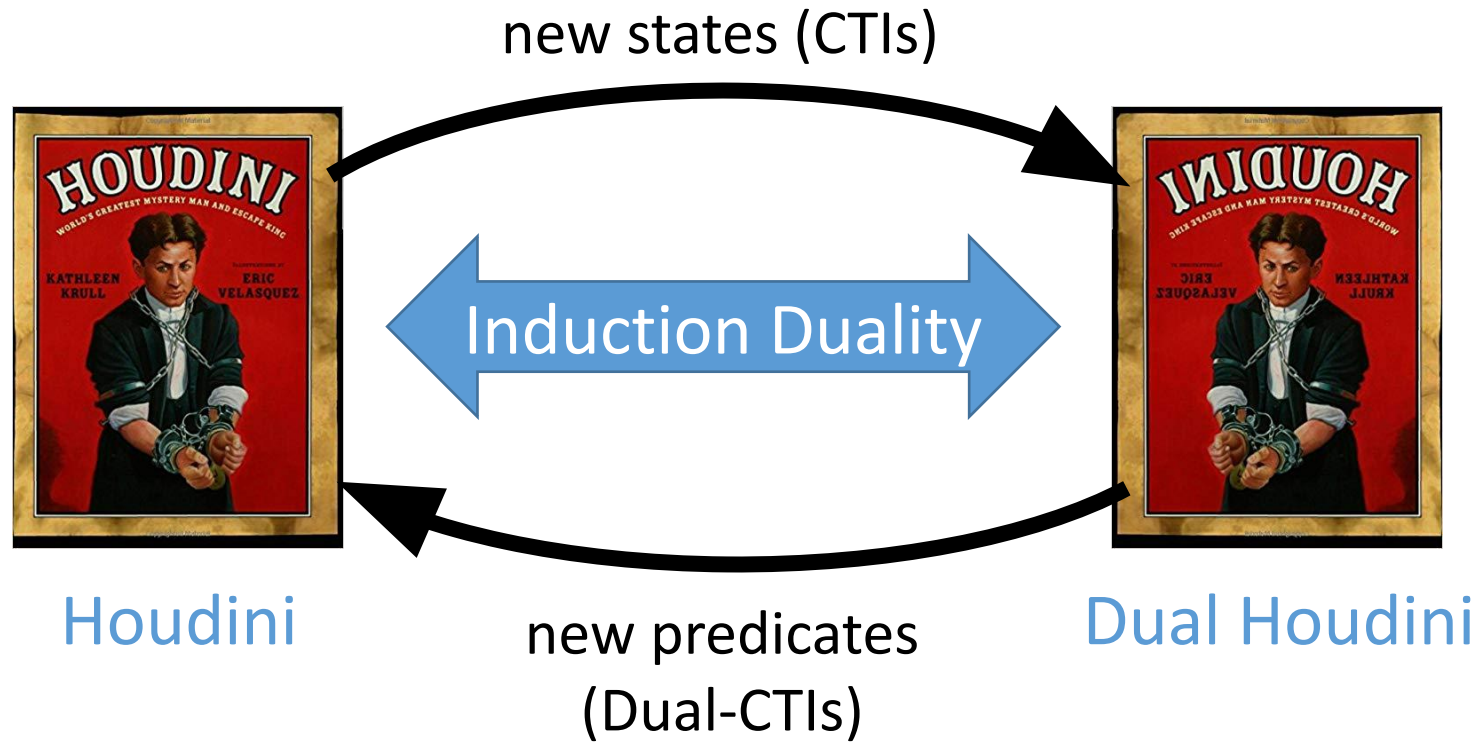
Concept	Induction dual	$\xleftrightarrow[\alpha]{\gamma}$ dual
reachable: E^*	k-provable	invariant
k-provable: E'^*	reachable	k-abstractly-reachable
invariant: $\alpha(E^*)$	k-abstractly-reachable	reachable
k-abstractly-reachable: $\gamma(E'^*)$	invariant	k-provable
inductive: E'_ω	dual-inductive	$\supseteq$ reachable
dual-inductive: E_ω	inductive	$\supseteq$ k-provable

Houdini and Dual Houdini



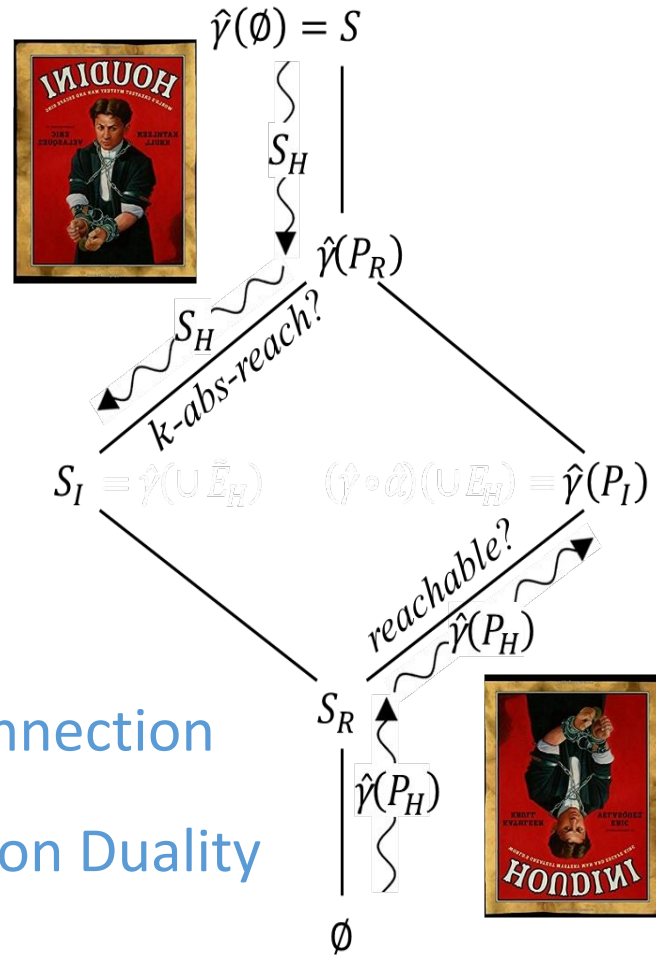
Theorem: all the states in S_H cannot be ruled out by k -bounded incremental induction (no matter how long)

Primal-Dual Houdini

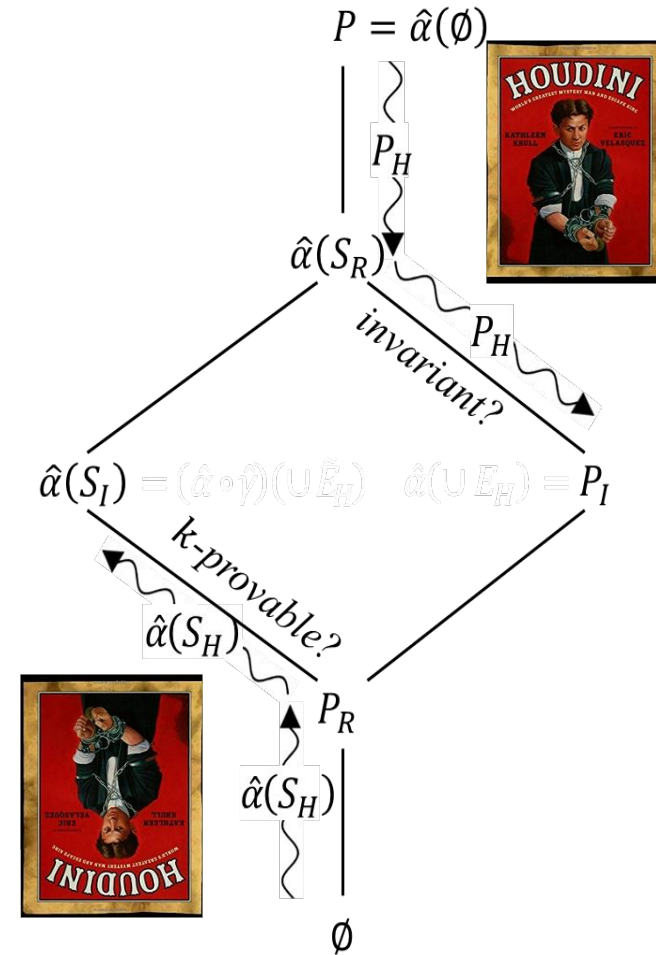


Galois Connection and Induction Duality

Lattice of sets
of states



Lattice of sets
of predicates



Vertical flip: Galois Connection
Horizontal flip: Induction Duality

Example: Toy Consensus Protocol

- v (Node, Value) – votes
- b (Node) – bit used by a node to remember if it voted already
- d (Value) – decisions
- m (Node, Quorum) – membership

axiom $\forall q_1, q_2: \text{Quorum}. \exists n: \text{Node}. m(n, q_1) \wedge m(n, q_2)$

action $\text{vote}(n: \text{Node}, x: \text{value})$
requires $\neg b(n)$
 $v(n, x) := \text{true}$
 $b(n) := \text{true}$

action $\text{decide}(x: \text{value})$
requires $\exists q \forall n. m(n, q) \rightarrow v(n, x)$
 $d(x) := \text{true}$

Safety specification:

$$\forall x, y. d(x) \wedge d(y) \rightarrow x = y$$

Inductive invariant:

$$\forall x. d(x) \rightarrow \exists q \forall n. m(n, q) \rightarrow v(n, x)$$

$$\forall n, x, y. v(n, x) \wedge v(n, y) \rightarrow x = y$$

$$\forall n, x. v(n, x) \rightarrow b(n)$$

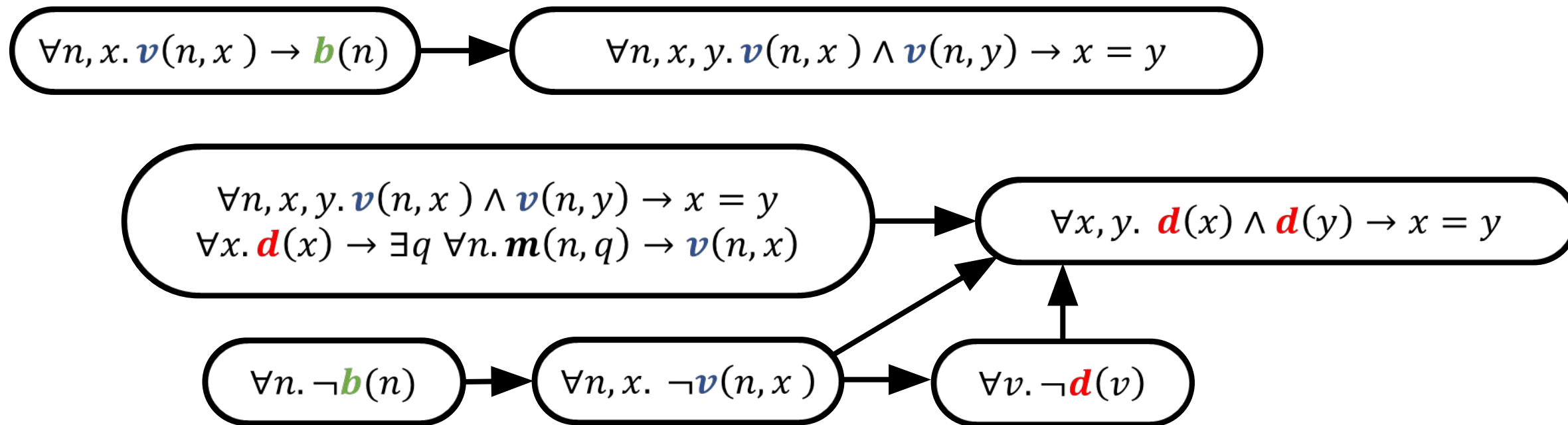
Induction Edges

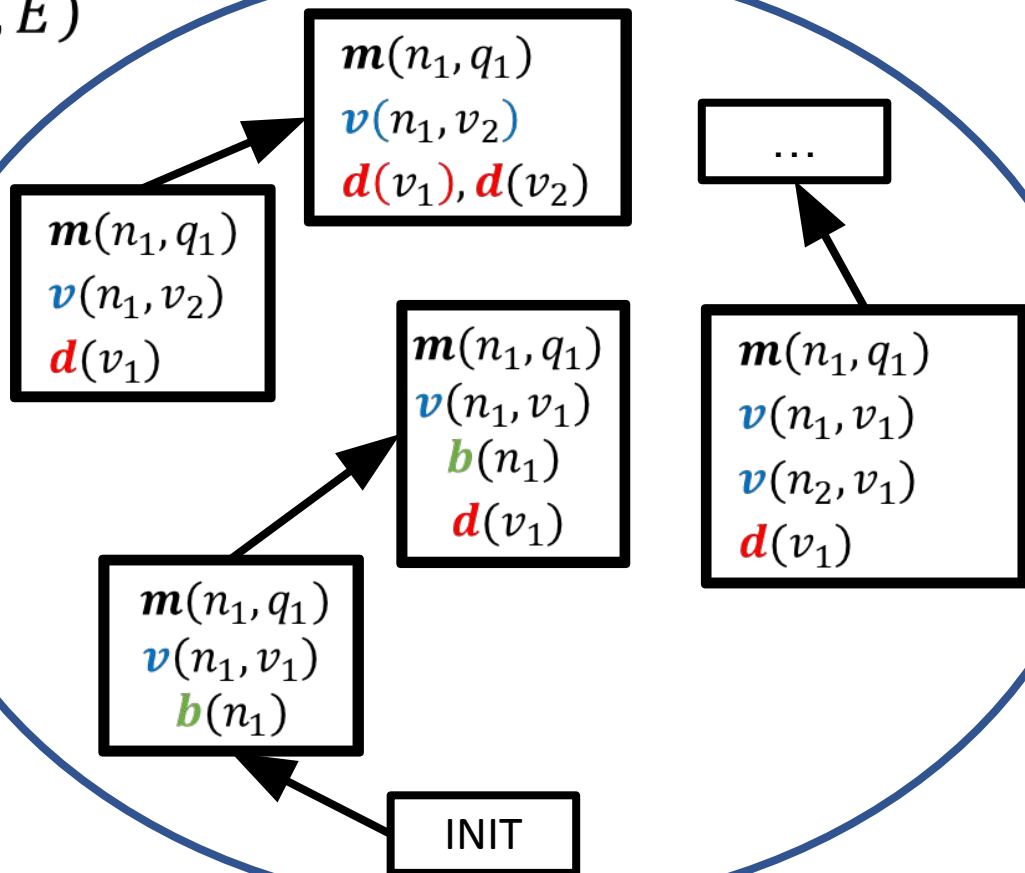
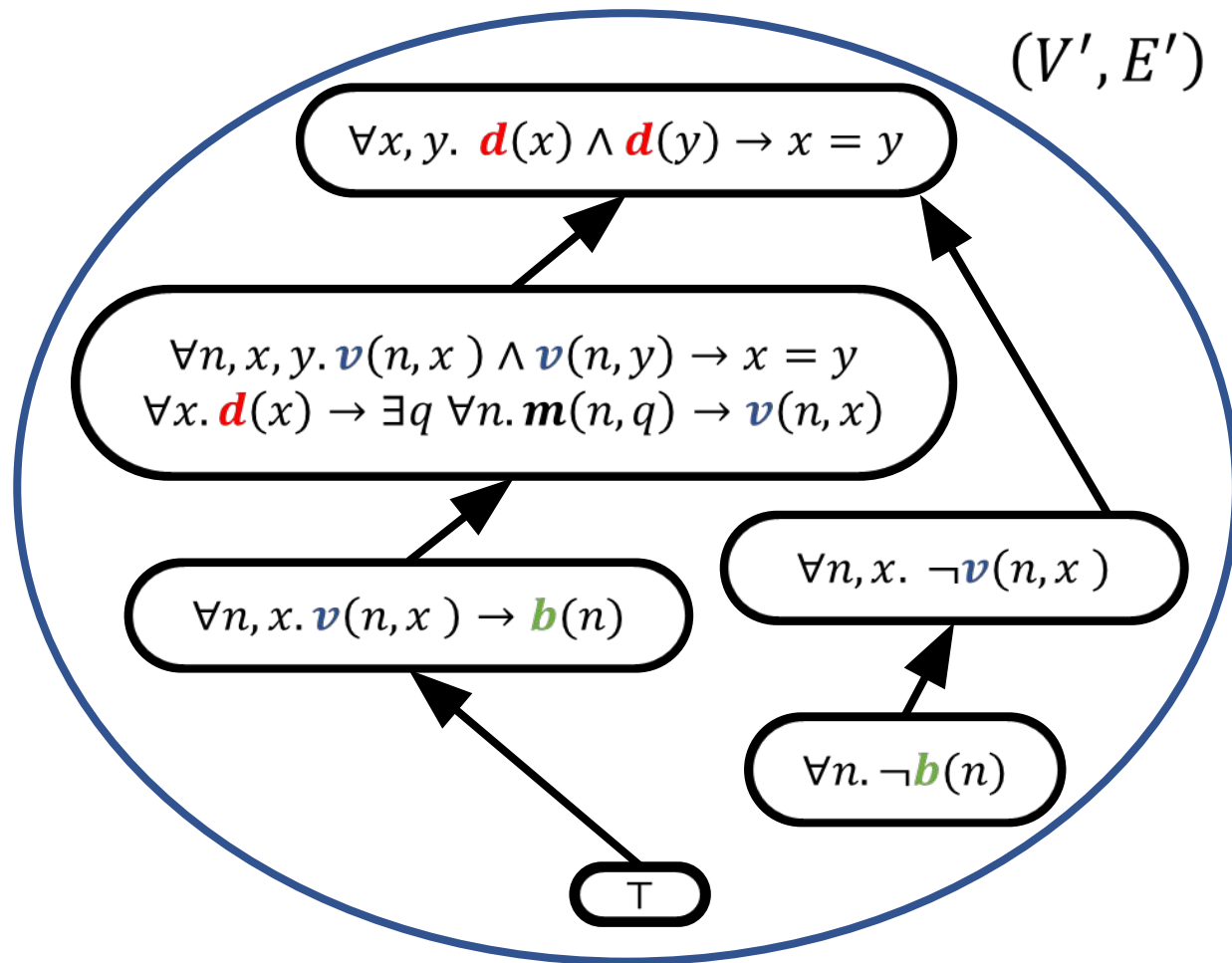
- Induction edge from P to Q if:

$$\text{post}([P] \cap [Q]) \cap [P] \subseteq [Q] \quad \Leftrightarrow \quad P \wedge Q \models \text{wp}(P \rightarrow Q)$$

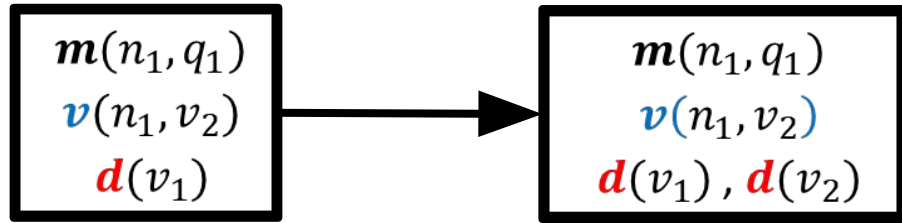
- Intuition: if P is invariant, Q is also invariant

- If a trace violates Q, it must also violate P (no later than the violation of Q)



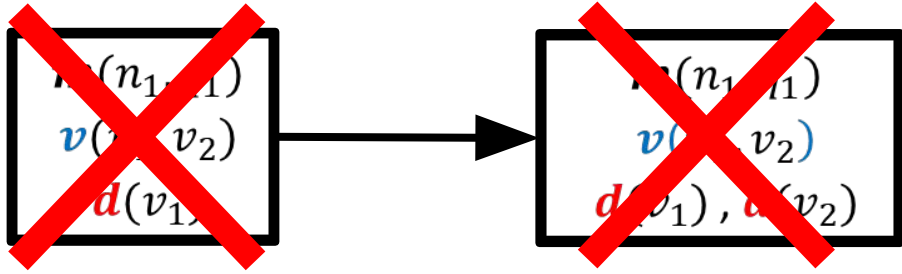
(V, E)  (V', E') 

Primal Houdini



A logical formula is shown inside a rounded rectangular box, which is crossed out with a large red X. The formula is: $\forall v_1, v_2. d(v_1) \wedge d(v_2) \rightarrow v_1 = v_2$

Dual Houdini

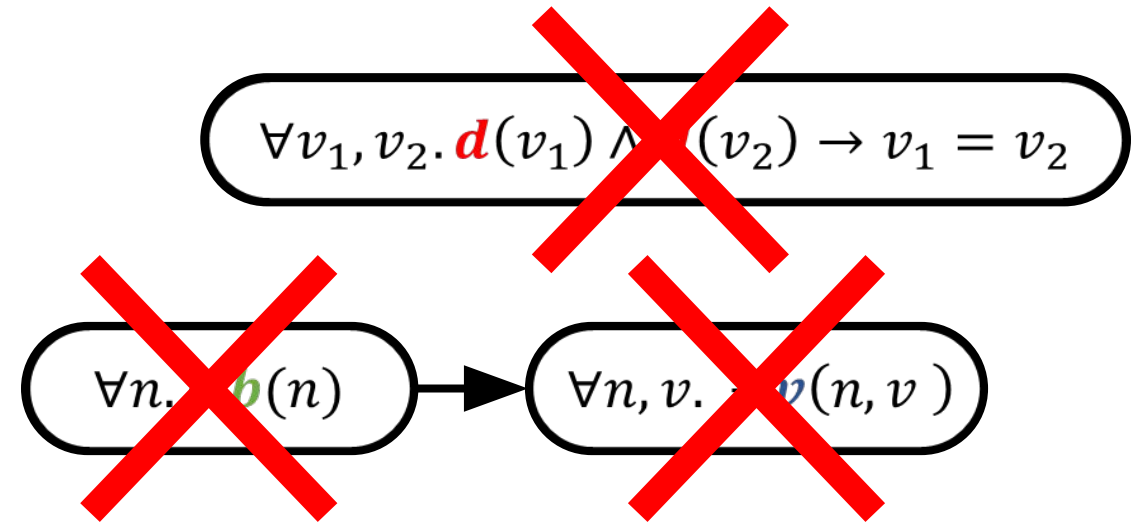
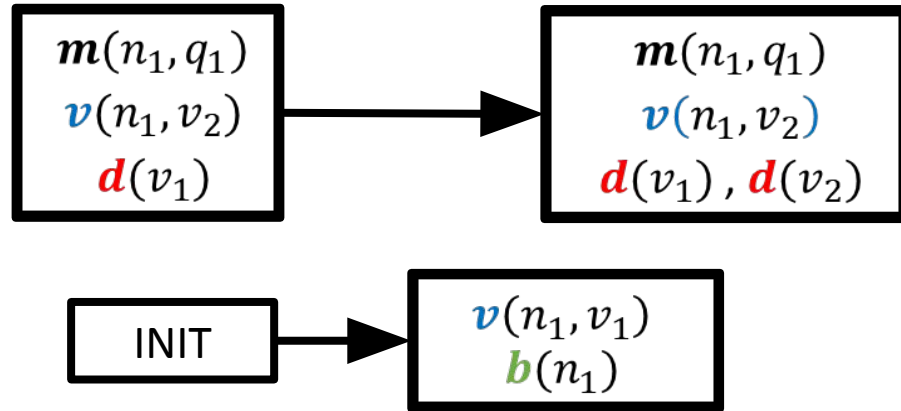


$$\forall v_1, v_2. \textcolor{red}{d}(v_1) \wedge \textcolor{red}{d}(v_2) \rightarrow v_1 = v_2$$

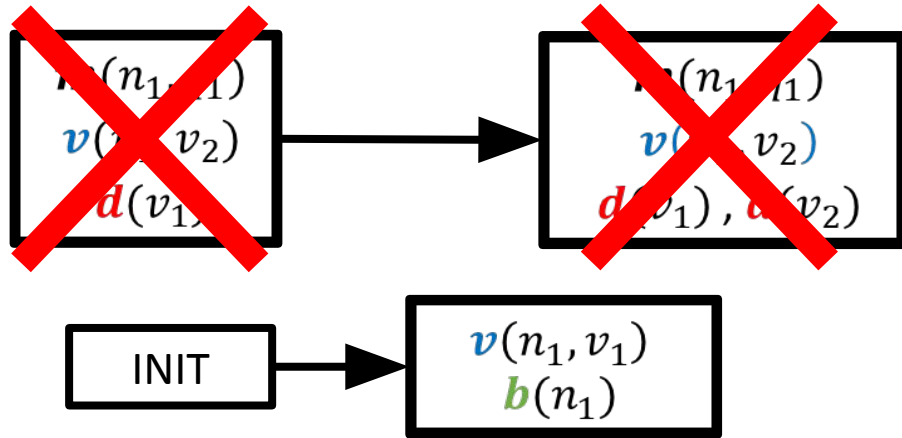
$$\forall n. \neg \textcolor{green}{b}(n)$$

$$\forall n, v. \neg \textcolor{blue}{v}(n, v)$$

Primal Houdini



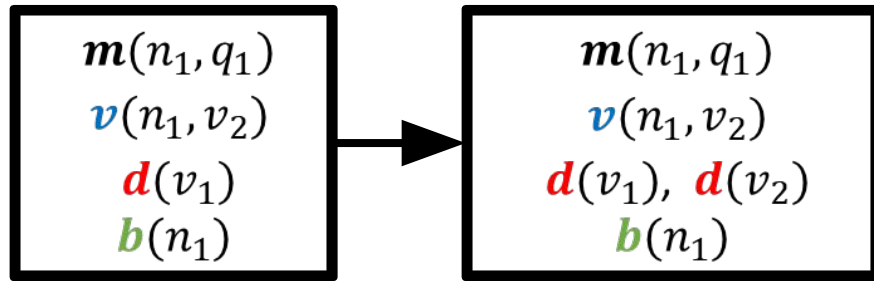
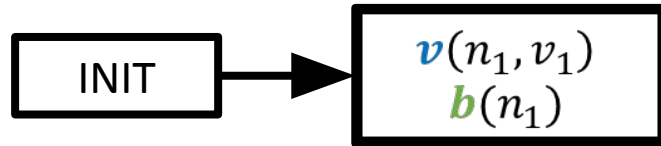
Dual Houdini



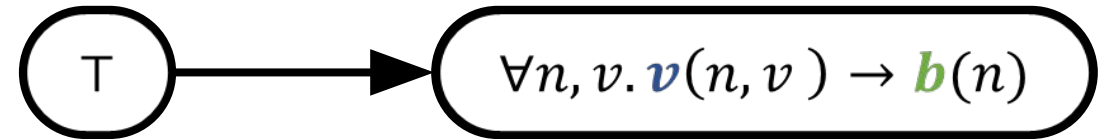
$$\forall v_1, v_2. \mathbf{d}(v_1) \wedge \mathbf{d}(v_2) \rightarrow v_1 = v_2$$

T $\rightarrow \forall n, v. \mathbf{v}(n, v) \rightarrow \mathbf{b}(n)$

Primal Houdini

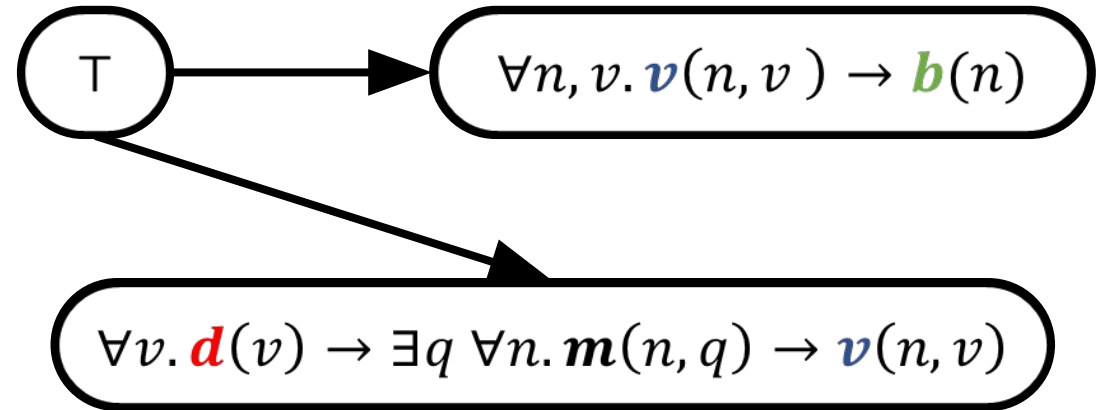
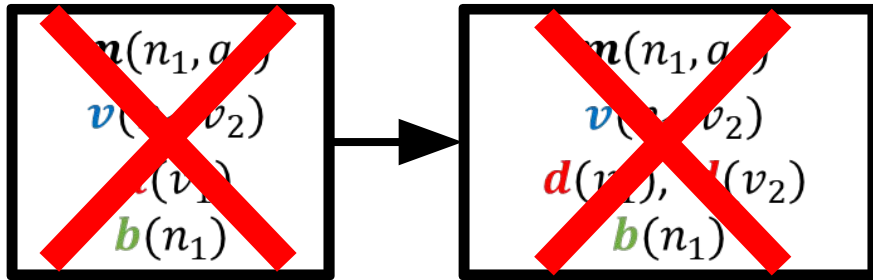
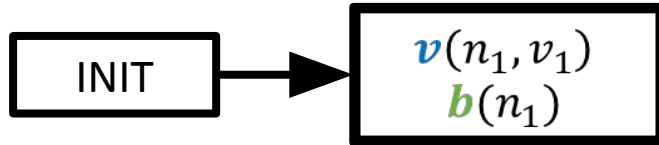


$$\forall v_1, v_2. d(v_1) \wedge d(v_2) \rightarrow v_1 = v_2$$

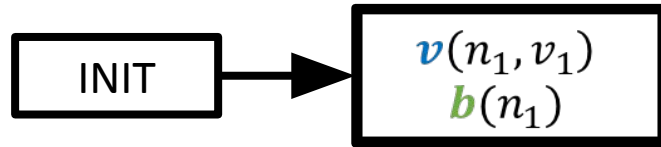


Dual Houdini

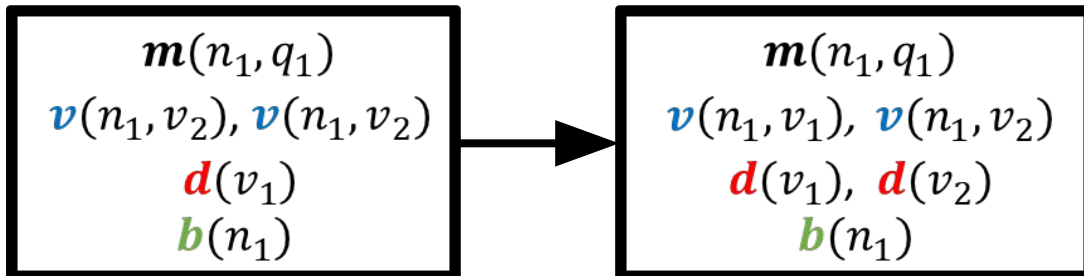
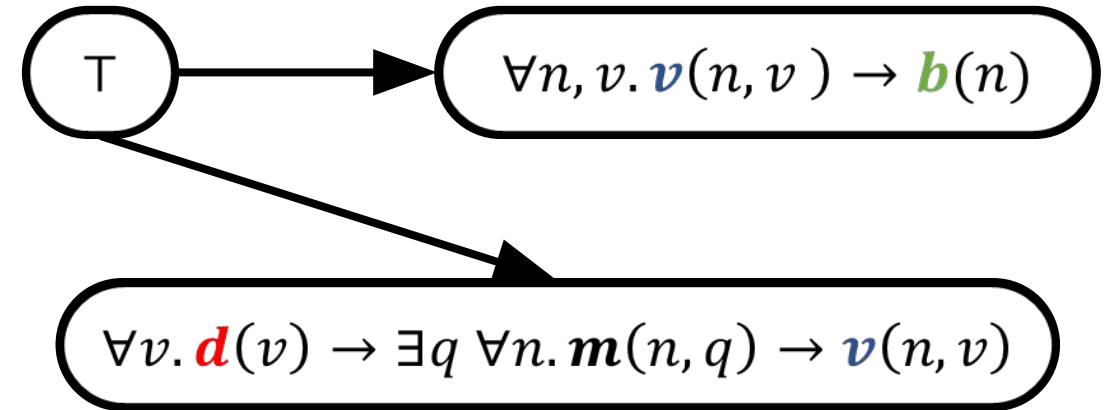
$$\forall v_1, v_2. \textcolor{red}{d}(v_1) \wedge \textcolor{red}{d}(v_2) \rightarrow v_1 = v_2$$



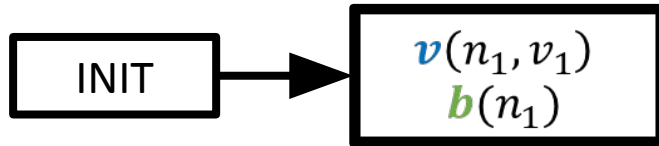
Primal Houdini



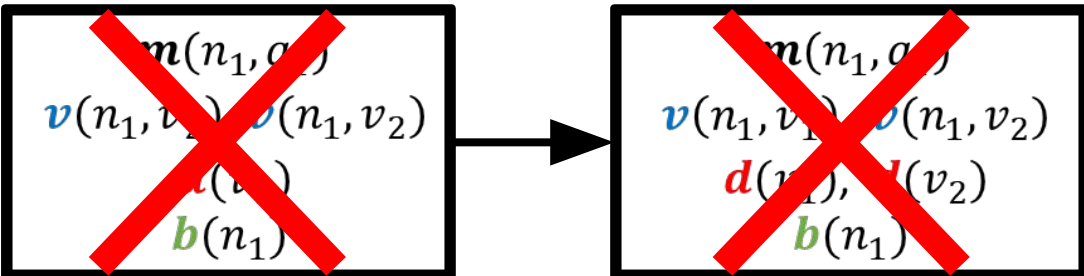
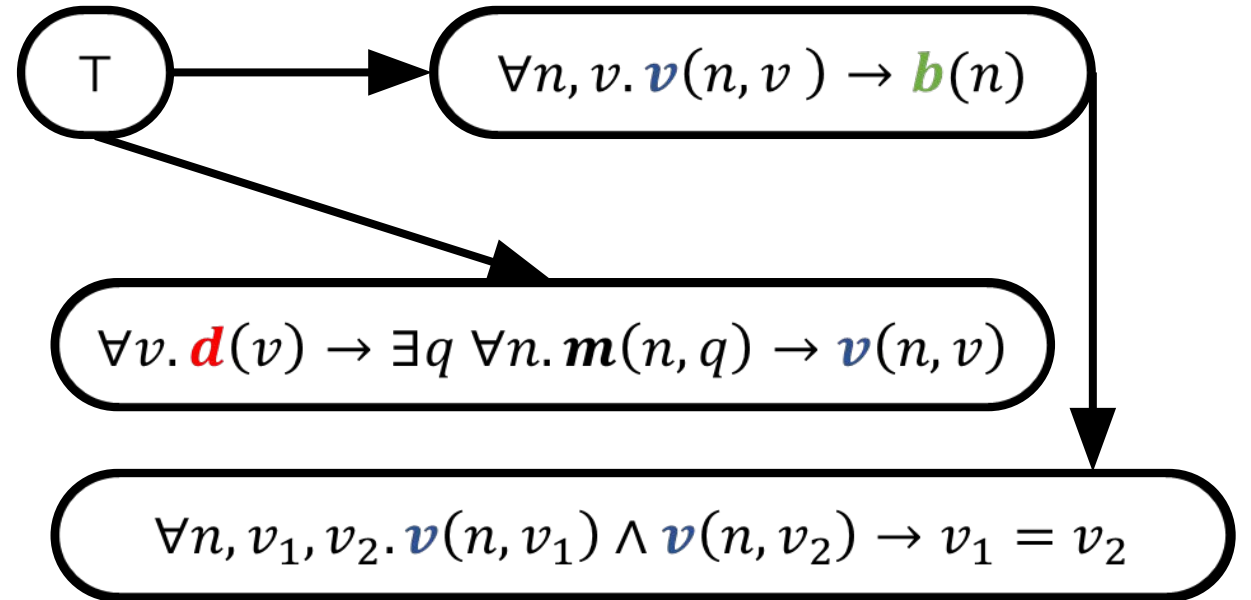
$$\forall v_1, v_2. \textcolor{red}{d}(v_1) \wedge \textcolor{red}{d}(v_2) \rightarrow v_1 = v_2$$



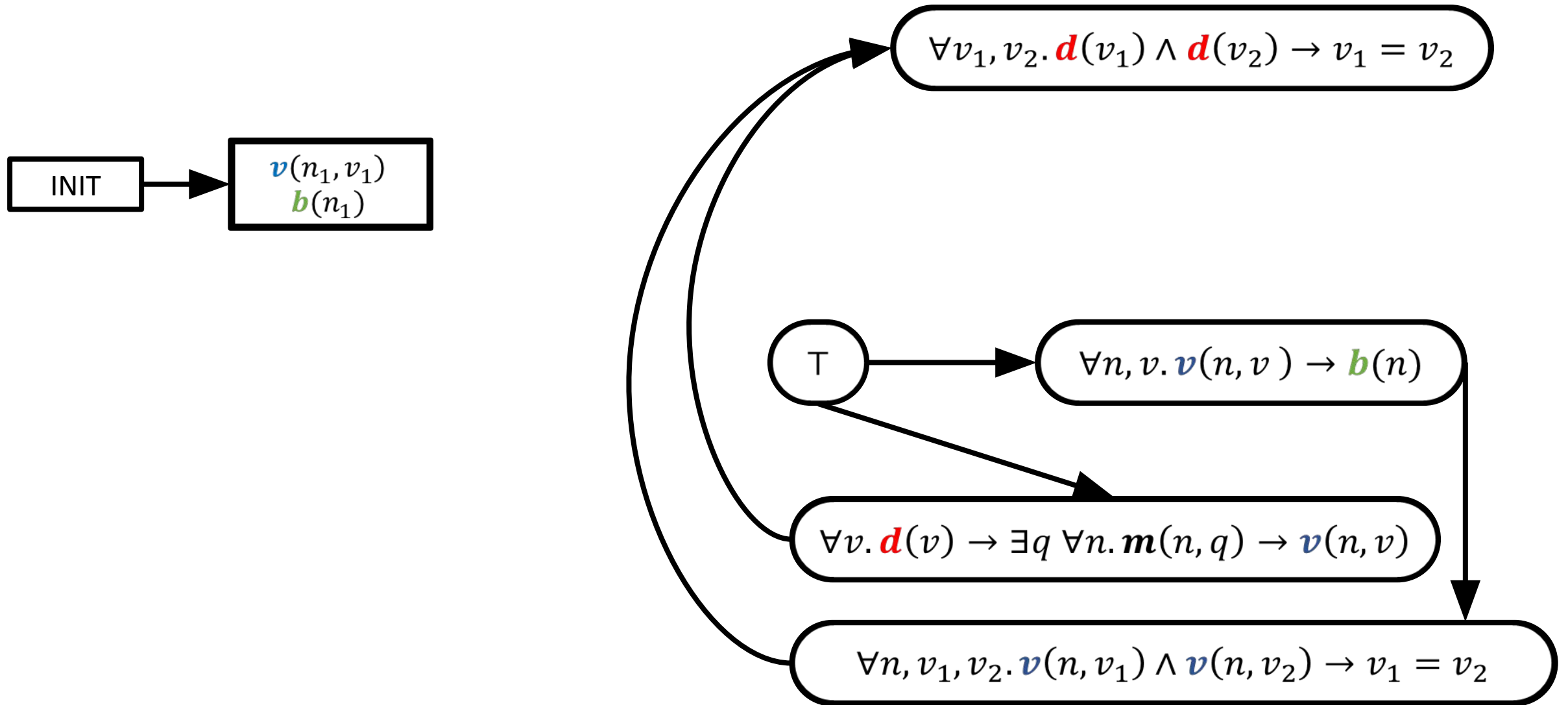
Dual Houdini



$$\forall v_1, v_2. \textcolor{red}{d}(v_1) \wedge \textcolor{red}{d}(v_2) \rightarrow v_1 = v_2$$



Primal Houdini



Primal-Dual Houdini: Theorems

- **Exploration:** every iteration discovers new states and predicates
- **Possible Progress:**
 - If k-provable, every iteration can discover a new useful predicate
 - If unsafe, every iteration can discover a new useful state
- **Termination by Stratification:** if states and predicates are discovered in a stratified manner, then the algorithm terminates for k-provable or unsafe cases

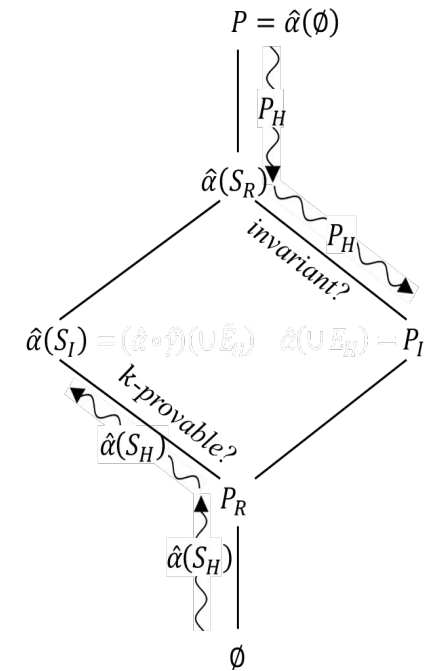
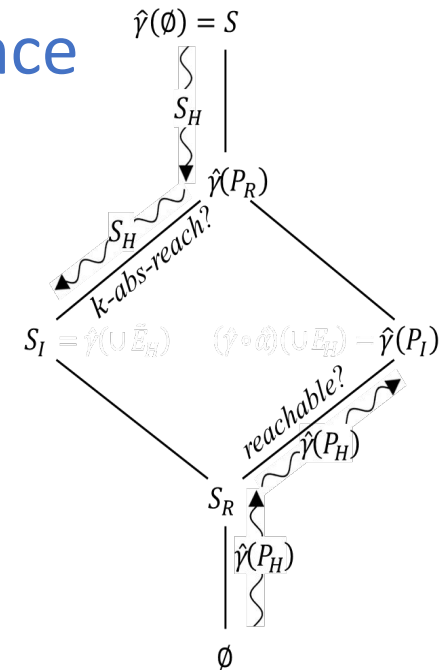
Evaluation for Distributed Protocols ($k = 1$)

Example	pdH	UPDR	FOL-IC3	SWISS	IC3PO	DistAI
ring	✓	✓	✓	✓	✓	✓
cons	✓	✓	✓	✓	✓	✓
paxos	✓	✓		✓		
spaxos	✓				✓	
paxos-h	✓					
spaxos-h						
locksrv	✓	✓	✓	✓	✓	✓
skv		✓	✓	✓	✓	✓
skvr		✓				
cache					✓	

[POPL'22] O. P., James R. Wilcox, Jason R. Koenig, Kenneth L. McMillan, and Alex Aiken.
Induction Duality: Primal-Dual Search for Invariants.

Conclusion (Primal-dual Houdini)

- Goal: symmetric primal-dual invariant inference
- Key idea: bounded incremental induction
- Result 1: Induction duality
- Result 2: Primal-Dual Houdini
- New synthesis task: check dual-inductiveness and find induction edges
- Future directions
 - Application to more domains
 - Primal-dual version of more advanced algorithms



Conclusion

- Using undecidable reasoning is powerful, but at a price
- Decidable logic (EPR) offers a different tradeoff
 - More effort in encoding systems and properties
 - More reliable automation
- Reliable invariant checking opens the path to invariant inference
 - Many recent ideas (and new adaptations of old ideas)
 - We are exhausting the benchmark set – progress would come from new benchmarks!
- Induction duality
 - Promising new idea
 - May be applicable in other domains
- Reach out to discuss more or collaborate!

